

The Smartest Way to Learn Python Regular Expressions

Learn the Best-Kept Productivity Secret
of Code Masters

Christian Mayer, Lukas Rieger, and Zohaib Riaz

Table of Contents

Table of Contents	ii
1 Introduction	1
2 Applications	7
3 About	24
4 Puzzle Learning	28
5 Basics	35
6 Special Symbols	44
7 Character Sets	53

TABLE OF CONTENTS

iii

8	Dot Regex	68
9	Asterisk Quantifier	81
10	Plus Quantifier	87
11	? Quantifier	94
12	Quantifier Differentiation	102
13	Greediness	106
14	Line and String Boundaries	119
15	OR Regex	132
16	AND Regex	140
17	NOT Regex	148
18	Matching Groups	151
19	Split Method	167
20	Substitution Method	175
21	Compile Method	185
22	Bonus Puzzles	195

23 Final Remarks**224**

Long Table of Contents

Table of Contents	ii
1 Introduction	1
2 Applications	7
3 About	24
4 Puzzle Learning	28
5 Basics	35
5.1 What's a Regex Pattern?	36
5.2 The Method to Find All Matches . . .	40
5.3 Summary	42
6 Special Symbols	44

6.1	Literal Characters	44
6.2	Summary	51
7	Character Sets	53
7.1	What's a Character Set?	53
7.2	Negative Character Set	55
7.3	Summary	56
7.4	Intermezzo: Match Function	58
7.5	Methods <code>re.match()</code> vs <code>re.findall()</code>	62
7.6	<code>re.fullmatch()</code>	63
7.7	Methods <code>re.fullmatch()</code> vs <code>re.match()</code> .	66
8	Dot Regex	68
8.1	What's the Dot Regex?	68
8.2	Intermezzo: Flags	71
8.3	Match Newline	75
8.4	Match the Dot Character	76
8.5	Summary	78
9	Asterisk Quantifier	81
9.1	What's the Asterisk Quantifier?	81
9.2	Asterisk Example	82
9.3	Match the Asterisk Character	84
9.4	Summary	85
10	Plus Quantifier	87
10.1	What's the Plus Quantifier?	87

10.2	Plus (+) Quantifiers Examples	88
10.3	Match the Plus (+) Symbol	90
10.4	Summary	91
11	? Quantifier	94
11.1	Definition Question Mark	94
11.2	Summary	98
12	Quantifier Differentiation	102
12.1	Asterisk vs Plus	103
12.2	Asterisk vs Question Mark	103
	Question Mark vs Plus	104
13	Greediness	106
13.1	Regex Quantifiers	106
13.2	Greedy Match	109
13.3	Non-Greedy Match	110
13.4	Non-Greedy Question Mark (??)	111
13.5	Non-Greedy Asterisk (*?)	112
13.6	Non-Greedy Plus (+?)	112
13.7	Greedy vs Non-Greedy	113
13.8	Which is Faster?	114
13.9	Summary	117
14	Line and String Boundaries	119
14.1	Start-of-String (^)	119
14.2	Start-of-Line (^)	120

14.3	End of String (\$)	122
14.4	End of Line (\$)	122
14.5	Intermezzo: Regex Methods	124
14.6	Match Caret (^) or Dollar (\$) Symbols	128
14.7	Summary	129
15	OR Regex	132
15.1	What's the OR Operator?	132
15.2	Examples	133
15.3	Nested OR	135
15.4	Character Set OR	136
15.5	Match the Vertical Line ' '	137
15.6	Summary	138
16	AND Regex	140
16.1	Ordered Regex AND Operator	141
16.2	Unordered Regex AND Operator	142
16.3	Summary	145
17	NOT Regex	148
18	Matching Groups	151
18.1	Simple Matching Groups	151
18.2	First Matching Group	153
18.3	Other Matching Groups	154
18.4	Named Groups	156
18.5	Non-Capturing Groups	157

18.6	Positive Lookahead	159
18.7	Negative Lookahead	161
18.8	Group Flags	162
18.9	Summary	164
19	Split Method	167
19.1	re.split()	167
19.2	Minimal Example	169
19.3	Maxsplit Argument	170
19.4	Optional Flag Argument	171
19.5	Regex Versus String Split	172
19.6	Summary	173
20	Substitution Method	175
20.1	re.sub()	175
20.2	Minimal Example	177
20.3	count Argument	179
20.4	Optional Flag Argument	180
20.5	Regex Sub Versus String Replace . . .	181
20.6	Remove Regex Pattern	182
20.7	Summary	183
21	Compile Method	185
21.1	re.compile()	185
21.2	Regular Expression Object	187
21.3	Discussion	188
21.4	Summary	192

22 Bonus Puzzles	195
23 Final Remarks	224
Where to go from here?	225

— 1 —

Introduction

The human brain is a marvelous creation. Even with today's cutting-edge scientific technology, we understand very little about its detailed working. Putting aside the numerous body functions controlled by the brain, let's think for a moment about a single one: the ability to recall information from our memory. We know very well that the brain does this remarkably fast. However, it is not just the speed that is remarkable. It's also astounding how easy does the brain make it for us: we just have to think about the topic of interest. As soon as we do that, the relevant thoughts and memories start popping up in our minds.

Now compare this functionality of the human brain

with modern-day computers. For instance, if you wish to search a single word “accommodation” in a text file, you have to be careful in spelling it correctly in the search box of the underlying application (e.g., Notepad). If you happen to misspell it, the search may fail even if the text file contains numerous occurrences of “accommodation”. The point we want to highlight here is that, unlike the brain, computers always need precise instructions. This also explains why we have coding-languages, such as Python, as a medium of instruction from humans to machines.

However, it’s a shame that we must always be so “mechanical” when communicating with computers. It’s just not natural. In stark contrast to machines, we as humans can speak to each other quite *imprecisely*. Our high intelligence caters to the inaccuracies of our expression such that we can successfully exchange information and ideas by even talking *roughly* about them. For example, imagine traveling to a country whose language you do not know. You can still reasonably expect the local people to, for example, guide you to nearby restaurants if you approach them with some approximate expression of your need (e.g., a few words in the local language or even some sign language).

For computers to do the same, extra work is re-

quired to make them aware of the possible inaccuracies or deficiencies of human expression and to equip them to handle these appropriately. You can see a simple example of this extra work in today's online search engines, such as Google. These search-engines are generally intelligent enough to look up relevant web pages while handling misspelled or out-of-order search terms. On a high level, they present a much-desired, human-like interface where approximate search terms are mapped to quite precise topics occurring in relevant web pages. This human-like functionality has a cost that not everyone can afford: it must be programmed into these search engines. If you and I want to implement such human-like functionality for offline searches on our PCs, its quite infeasible given the high levels of the required expertise as well as the effort.

This leads us (and probably you too) to ask the following questions: Is there an “easier” way of generally improving our day-to-day communication with computers, making it more human-like? We surely don't want to implement complex natural-language-processing algorithms¹ by hand. Even if we somehow

¹Modern day machine-learning techniques that enable computers to understand text information produced by humans, such as books or articles.

implement these algorithms, do we really understand how to benefit from their strengths maximally? Is it something that anybody could do? Or does it boil down to a “nerdy” topic that no non-expert should touch? And most of all, what’s bad about the mechanical nature of computers? For example, aren’t we already used to instructing them precisely (e.g., for searching stuff)? What improvement can we really make there, if any?

This book will try to address the above questions with the following motivation. We, the authors of this book, have a strong feeling that most people are not aware of existing tools that can remarkably (and really very remarkably!) improve human to computer communication. Tools that are not intended for coding-experts only, but rather represent the low hanging fruits that everyone can benefit from with little effort. In our opinion, these tools can boost a novice coder’s skill-set so much so that they can make expert coders look like fools. In other words, an expert coder who is ignorant of these tools may need to write numerous lines of code to achieve the same result as a novice coder (who knows these tools) would achieve with a single line of code!

Without further ado, the specific tools that we introduce in this book are called Regular Expressions.

A regular expression, also referred to as “regex” or “regexp” for short, is basically a string that defines a search pattern. In comparison to the usual “exact” search-strings, regexes are highly flexible and can trivially accommodate *in-exact* and *imprecise* search patterns. At the same time, they are very powerful in that they allow you to perform searches for complex patterns that otherwise would take significant, often iterative effort when using exact search-strings.

Learning to write these one-liner regexes should tremendously improve how fast you can look up information. Note that the ability to look up the right information (that you really want to find) is not a trivial one. It’s a distinguishing gift!

For anyone dealing with any form of text data, be it a data-scientist collecting and analyzing data-sets, or a software engineer writing or even just exploring huge code-bases, knowing how to use regexes can boost their efficiency and capability drastically.

We, the authors of this book, would like to emphasize again that we firmly believe that regexes are one the most under-rated tools whose usefulness has been unjustifiably overlooked. In our humble opinion, we rate regexes as the second most important skill to learn, right after basic typing skills.

Throughout this book, we will try to keep convincing you in favor of using regexes in your daily work by presenting you with diverse and motivating use-cases. With the multi-faceted, engaging style of teaching adopted in this book (write-up, code-puzzles, and videos), we hope that you will find it easier to appreciate the usefulness of regexes.

To stretch your imagination already, we will now describe several applications where regular expressions are employed.

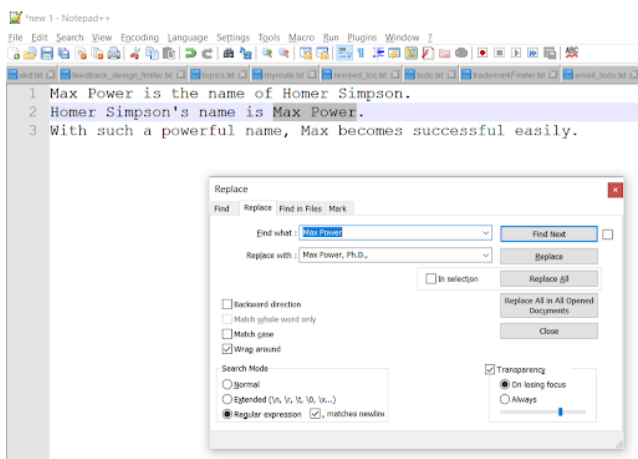
Applications

Regexes are widely used in many practical applications. Let's study some of them in this section.

Search and Replace in a Text Editor: You'll use regular expressions to search a given text in a text editor. Say, your boss asks you to replace all occurrences of a customer **Max Power** with the name **Max Power, Ph.D.,**. How'd you do this? In figure 2.1, we show how it would look like in the popular text editor Notepad++ (recommended for coders).

At the bottom of the “Replace” window, you can see the “Regular expression” radio-button. When developing your regular expression, you realize that you also want to replace all occurrences of **Max** (without the surname **Power**). To accomplish that, you use a

Figure 2.1: Screenshot of a normal text search.

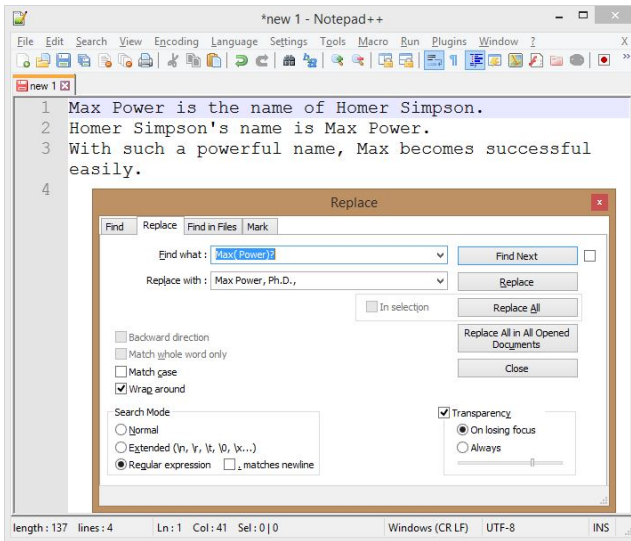


regex in the search field: `Max( Power)?` rather than only `Max Power` (see Figure 2.2).

For now, you don't need to worry about the specific regex `Max( Power)?` and why it works for this task. You just need to know that it's possible to match all occurrences of either `Max Power` or `Max`.

Counting words: How to count the number words in a text document? You could copy the text into a new Microsoft Word document and check the word count on the bottom left. However, what if you don't

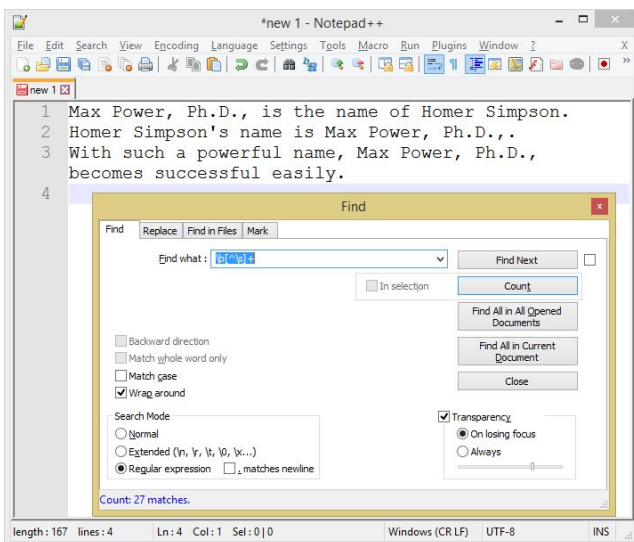
Figure 2.2: Screenshot of a regex text search.



have a Microsoft Word license for your machine, or you simply want to do it somehow inside the Notepad++ editor. Well, as you may have guessed, there is a way by using regular expressions. With the “Regular expression” option selected under the “find” tab, you enter the following regex in the search field: `\b[^\s]+`. Then press the “Count” button. Voila!, the number of words (27) is displayed at the bottom of the search window (see Figure 2.3). Again, you don’t need to

worry about the working of the regex `\b[^\s]+`. We will make sure that the later chapters of this book make things crystal clear!

Figure 2.3: Screenshot of a regex for word-count.



Searching Your Operating System for Files:

You can also use regular expressions to search (and find) specific files on your operating system. For example, a forum user tried to find all files with the following filename patterns:

```
abc.txt.r12222  
tjy.java.r9994
```

You can find all those files on Windows using the command:

```
dir * /s/b | findstr \.r[0-9]+$
```

Large parts of the commands are a regular expression. In the final part, you require the file to end with `.r` and an arbitrary number of numeric symbols. As soon as you've mastered regular expressions, this will cost you no time at all, and your productivity with your computer will skyrocket.

Searching Your Files for Text

But what if you don't want to find files with a specific filename but with a particular file content? Isn't this much harder? As it turns out, it isn't! Well, if you use regular expressions and `grep`. Here's what a `grep` guru would do to find all lines in a file `'haiku.txt'` that contain the word `'not'`.

```
$ grep not haiku.txt
```

This may lead to the following output of all lines that contain the word `'not'`:

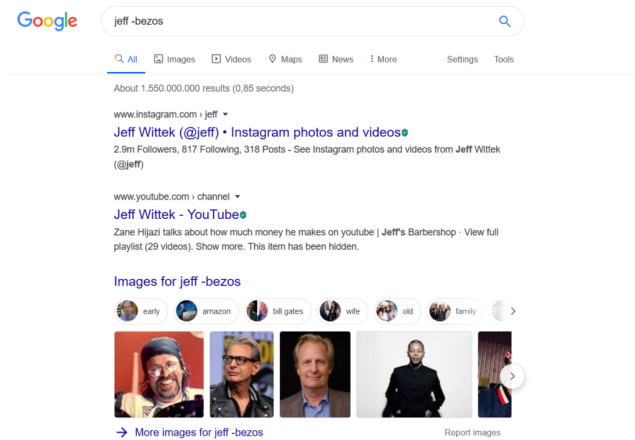
```
Is not the true Tao, until  
"My Thesis" not found  
Today it is not working
```

Grep is an age-old file search tool written by famous computer scientist Ken Thompson. And it's even more powerful than that: you can also search a bunch of files for specific content. A Windows version of grep is the find command.

Search Engines: Using regular expressions to find content on the web is considered the holy grail of search. But the internet is a huge beast, and supporting a full-fledged regex engine would be too demanding for Google's servers. It costs a lot of computational resources. Therefore, nobody actually provides a search engine that allows all regex commands. However, web search engines such as Google support a limited number of regex commands. For example, you can search queries that do NOT contain a specific word (see Figure 2.4).

The search "Jeff -Bezos" will give you all the Jeff's that do not end with Bezos. If a first name is dominated like this, using advanced search operators is quite a useful extension. Mastering search is a critical skill in the 21st century, after all.

Figure 2.4: A Google Search using the NOT operator.



Validate User Input in Web Applications: If you're running a web application, you need to deal with user input. Often, users can put anything in the input fields, even cross-site scripts to hack your webserver. Therefore, your application must validate the user input—otherwise, you're guaranteed to crash your backend application or database. How can you validate user input? Regex to the rescue!

Here's how you'd check whether

- The user input consists only of lowercase letters:

`[a-z]+`,

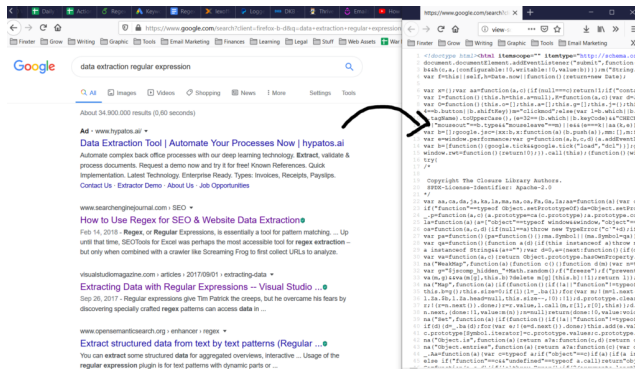
- The username consists of only lowercase letters, underscores, or numbers: `[a-z_0-9]+`, or
- The input does not contain any parentheses: `[^\(\)]+`.

With regular expressions, you can validate any user input—no matter how complicated it may seem. Think about this: any web application that processes user input needs regular expressions. Google, Facebook, Baidu, WeChat—all of those companies work with regular expressions to validate their user input. This skill is wildly important for your success as a developer working for those companies (or any other web-based company for that matter).

Web Crawlers to Extract Useful Information: Okay, you can validate user input with regular expressions. But is there more? You bet there is. Regular expressions are not only great to validate textual data but to extract information from textual data. For example, say you want to gain some advantage over your competition. You decide to write a web crawler that works 24/7 exploring a subset of webpages. A webpage links to other webpages. By going from webpage to webpage, your crawler can explore

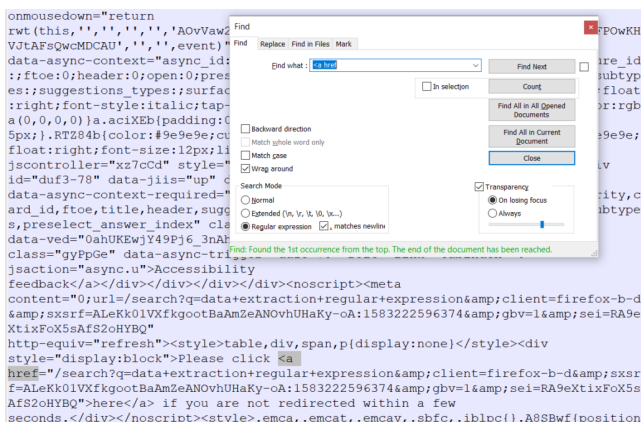
large parts of the web—fully automatized. Imagine the potential! Data is the asset class of the 21st century, and you can collect this valuable asset with your own web crawler. A web crawler can be a Python program that downloads the HTML content of a website (see Figure 2.5).

Figure 2.5: A Google Search.



Now, your crawler can use regular expressions to extract all outgoing links to other websites (starting with "[A simple regular expression can now automatically get the stuff that follows—which is the outgoing URL. You can store this URL in a list and visit it at a later point in time. As you're extracting links, you can](, see Figure 2.6).</p>
</div>
<div data-bbox=)

Figure 2.6: A Google Search using the NOT operator.



build a web graph, extract other information (e.g., embedded opinions of people), and run complicated subroutines on parts of the textual data (e.g., sentiment analysis). Don't underestimate the power of web crawlers when used in combination with regular expressions!

Data Scraping and Web Scraping: In the previous example, you've already seen how web crawlers benefit from regular expressions. But often, the first step is to simply download a specific type of data from a large number of websites to store it in a database (or a spreadsheet). The process of extracting a spe-

cific type of data from a set of websites and converting it to the desired data format is called *web scraping*. Web scrapers are needed in finance startups, analytics companies, law enforcement, eCommerce companies, and social networks. Regular expressions help much in processing the messy textual data. There are many different applications, such as finding titles of a bunch of blog articles (e.g., for search engine optimization). A minimal example of using Python's regex library `re` for web scraping is the following:

```
from urllib.request import urlopen
import re

url =
↳ "https://blog.finxter.com/start-learning-python/"
html = urlopen(url).read()

print(str(html))
titles = re.findall("<title>(.*?)</title>",
↳ str(html))

print(titles)
# ["What's The Best Way to Start Learning Python? A
↳ Tutorial in 10 Easy Steps! | Finxter"]
```

You extract all data that's enclosed in opening and closing title tags: `<title>...</title>`.

Data Wrangling: Data wrangling is the process of transforming raw data into a more useful format to simplify the processing of downstream applications. Such formatting of data and its subsequent filtering or “cleaning” is essential for nearly all data science and machine learning applications. As you may have guessed already, data wrangling is highly dependent on tools such as regular expression engines. In the upcoming chapters of this book, we will show you how to convert a string into a new one where a replacement string replaces each occurrence of pattern. This way, you can transform currencies, dates, or stock prices into a standard format with regular expressions.

Parsing: Any parser leverages hundreds of regular expressions to process the input quickly and effectively. You may ask: what’s a parser anyway? And you’re right to ask. A parser translates a string of symbols into a higher-level abstraction such as a formalized language, often using an underlying grammar to “understand” the symbols. You’ll need a parser to write your own programming language, syntax system, or text editor. For example, if you write a program in the Python programming language, it’s just a bunch of characters. Python’s parser brings order into the chaos and translates the characters from your code file into more meaningful abstractions (e.g., keywords,

variable names, or function definitions). This is then used as an input for further processing stages, such as the execution of your program. If you're looking at parser implementations, you'll see that they heavily rely on regular expressions. This makes sense because a regular expression can easily analyze and catch parts of your text. For example, to extract function names you can use the following regex in your parser:

```
import re

code = '''
def f1():
    return 1

def f2()
    return 2
'''

print(re.findall('def ([a-zA-Z0-9_]+)', code))
# ['f1', 'f2']
```

You can see that our mini parser extracts all function names in the code. Of course, it's only a minimal example, and it wouldn't work for all instances. For example, you can use more characters than the given ones to define a function name. If you're interested in writing parsers or learning about compilers, reg-

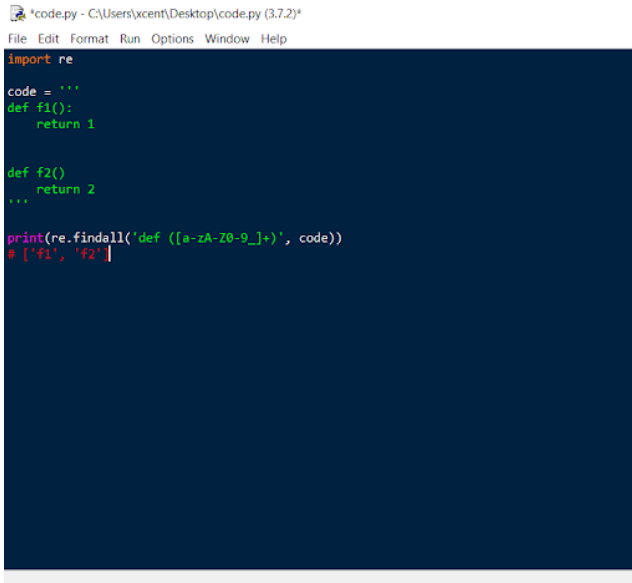
ular expressions are among the most useful tools in existence!

Programming Languages: Yes, you’ve already learned about parsers in the previous point. And parsers are needed for any programming language. Nearly all programming languages rely on regular expressions for their own implementation. For compiled languages, the compiler also uses regular expressions for lexical analysis to scan valid tokens that belong to the programming language to be compiled. The regular expressions define the grammar of the programming language (denoted as *regular grammar*. But regular expressions are also prevalent when writing code *within* the programming language. Some programming languages such as Perl provide built-in regex functionality: you don’t even need to import an external library. If you’re going to become an expert coder, you will use regular expressions in many coding projects. And the more you use it, the more you’ll learn to love and appreciate the power of regular expressions.

Syntax Highlighting Systems: Figure 2.7 shows how your standard coding environment may look like.

Any code editor provides syntax highlighting capabilities. Here’s an example:

Figure 2.7: Syntax Highlighting.



The image shows a screenshot of a Python code editor window. The title bar at the top reads "code.py - C:\Users\xcent\Desktop\code.py (3.7.2)". Below the title bar is a menu bar with the options: File, Edit, Format, Run, Options, Window, and Help. The main editing area has a dark blue background and displays Python code with syntax highlighting. The code is as follows:

```
import re
code = '''
def f1():
    return 1

def f2()
...
    return 2

print(re.findall('def ([a-zA-Z0-9_]+)', code))
# ['f1', 'f2']
```

In this code, the function names 'f1' and 'f2' are highlighted in blue. The strings 'code' and the list elements 'f1' and 'f2' are highlighted in yellow. The comment line starting with '#' is highlighted in red. All other code is white.

- Function names are blue.
- Strings are yellow.
- Comments are red.
- And normal code are white.

This way, reading and writing code becomes far more convenient. More advanced IDEs such as PyCharm provide dynamic tool-tips as an additional feature. Those are implemented with regular expressions to locate keywords, function names, and normal code snippets—in order to, ultimately, highlight them and enrich them with additional information, e.g., for tool-tips.

Formal Language Theory: Theoretical computer science is the foundation of all computer science. The great names in computer science, Alan Turing, Alonzo Church, and Steven Kleene spent significant time and effort studying and developing regular expressions. If you want to become a great computer scientist, you need to know your fair share of theoretical computer science. You should know the basics of formal language theory and regular expressions that are at the heart of these theoretical foundations.

How do regular expressions relate to formal language theory? Each regular expression defines a “language” of acceptable words. All words that match the regular expression are in this language. All words that do not match the regular expression are not in this language. This way, you can create a precise set of rules to describe any formal language—by expressing it in terms of the right regular expressions.

The applications of regexes that we described in this chapter are only a small subset of the ones used in practice. We hope to have given you a glance into how vital regular expressions have been, are, and will remain in the future.¹

Watch the short chapter video on YouTube:



<https://youtu.be/v6oi3M1l7sw>

¹If you want to learn how to apply some of your newly acquired regex skills, check out the Finxter Freelancer webinar at <https://blog.finxter.com/webinar-freelancer/> for free.

About This Book: Structure and Instruction Methodology

The main driver for mastery is neither a character trait nor talent. Mastery comes from intense, structured training. The author Malcolm Gladwell formulated the famous rule of 10,000 hours after collecting research from various fields such as psychology and neurological science.¹ The rule states that if you have average talent, you will reach mastery in any discipline by investing approximately 10,000 hours of intense training. Bill Gates, the founder of Microsoft, reached mastery at a young age as a result of coding for more than 10,000 hours. He was committed and passionate about coding and worked long nights to

¹Malcolm Gladwell *Outliers: The Story of Success*

develop his skills. He was anything but an overnight success.

If you are reading this book, you are an aspiring coder, and you seek ways to advance your coding skills. Nurturing your ambition to learn will pay a rich stream of dividends to you and your family as long as you live. It will make you a respectable member of society, providing unique value to information technology, automation, and digitization. Ultimately, it will give you strong confidence. So keeping your ambition to learn intact is the one thing you must place above all else.

The Smartest Way to Learn Python Regex is the sixth book in the Finxter book series. But as a regular expression introduction, it stands on its own. The other books in the Finxter book series are:

- *Coffee Break Python*
- *Coffee Break Python Workbook*
- *Coffee Break NumPy*
- *Brain Games Python*
- *Python One-Liners*
- *The Smartest Way to Learn Python Regex*

This book aims to be a stepping stone on your path to becoming a Python master. It helps you to learn faster by making use of the established principles of good teaching. It offers you 15 to 25 hours of thorough Python training using one of the most efficient learning techniques, called *practice testing*. Investing this time will improve your skills to write, read, and understand Python source code even further.

The idea is that you break the big task of learning a topic into a series of small interactive steps. In particular, you'll follow the 3-step interactive training method:

- Step 1: You read a book section that explains all the needed concepts and information. After reading the book section, you'll know the theory and have a profound understanding of the basics.
- Step 2: You watch the associated video after completing each section. We'll give you a link to the <https://finxter.com> app where you'll also find the supporting free video. This way, you're strengthening your intuitive understanding of the material through repetition and multi-modal learning. To see and hear the same material explained by your personal teacher

will help you gain deeper insights into the theory.

Step 3: You solve the associated code puzzle on the <https://finxter.com> app link after watching the video. This will commit your theoretical understanding of the material into your memory through the scientifically proven method of *practice testing*.

The next chapter explains and motivates the advantages of the Finxter method of puzzle-based learning. If you already know about the benefits of puzzle-based learning from previous books, and you want to dive right into the technical content, feel free to skip the next chapter.

A Case for Puzzle-based Learning

Definition: A *code puzzle* is an educative snippet of source code that teaches a single computer science concept by activating the learner’s curiosity and involving them in the learning process.

Before diving into practical puzzle-solving, let’s first study some reasons why puzzle-based learning accelerates your learning speed and improves retention of the learned material. There is robust evidence in psychological science for each of these reasons.

Overcome the Knowledge Gap: The great teacher Socrates delivered complex knowledge by asking a sequence of questions. Each question built on

answers to previous questions provided by the student.

A good teacher opens a gap between their knowledge and the learner's. The knowledge gap makes the learner realize that they do not know the answer to a burning question. This creates tension in the learner's mind. To close this gap, the learner awaits the missing piece of knowledge from the teacher. Better yet, the learner starts developing their own answers. The learner *craves knowledge*.

Code puzzles open an immediate knowledge gap. When looking at the code, you first do not understand the meaning of the puzzle. The puzzle's semantics are hidden. But only you can transform the unsolved puzzle into a solved one.

Embrace the Eureka Moment: Humans are unique because of their ability to learn. Fast and thorough learning has increased our chances of survival. Thus, evolution created a brilliant biological reaction to reinforce learning. Your brain is wired to seek new information; it is wired to always process data, to always learn.

Did you ever feel the sudden burst of happiness after experiencing a eureka moment? Your brain releases endorphins, the moment you close a knowledge

gap. The instant gratification from learning is highly addictive, but this addiction makes you smarter. Solving a puzzle gives your brain instant gratification. Easy puzzles open small, hard puzzles, which open large knowledge gaps. Overcome any of them and learn in the process.

Divide and Conquer: Learning to code is a complex task. You must learn a myriad of new concepts and language features. Many aspiring coders are overwhelmed by complexity. They seek a clear path to mastery.

As any productivity expert will tell you: break a big task or goal into a series of smaller steps. Finishing each tiny step brings you one step closer to your big goal. *Divide and conquer* makes you feel in control, pushing you one step closer toward mastery. This book is the embodiment of divide and conquer: the big task of learning regular expressions is broken down into a series of smaller steps. In addition to that, each puzzle is a step toward your bigger goal of mastering computer science. Keep solving puzzles and you keep improving your skills.

Improve From Immediate Feedback: As a child, you learned to walk by trial and error—try, receive feedback, adapt, and repeat. Unconsciously, you

will minimize negative and maximize positive feedback. You avoid falling because it hurts, and you seek the approval of your parents. To learn anything, you need feedback such that you can adapt your actions.

An excellent learning environment provides you with *immediate* feedback for your actions. If you were to slap your friend each time he lights a cigarette—a not overly drastic measure to save his life—he would quickly stop smoking. Puzzle-based learning offers you an environment with immediate feedback to make learning to code easy and fast.

Small is Beautiful: The 21st century has seen a rise in microcontent. Microcontent is a short and accessible piece of valuable information such as the weather forecast, a news headline, or a cat video. Social media giants like Facebook and Twitter offer a stream of never-ending microcontent. Microcontent has many benefits: the consumer stays engaged and interested, and it is easily digestible in a short time. Each piece of microcontent pushes your knowledge horizon a bit further.

This book and its backend Finxter offers a stream of self-contained microcontent in the form of hundreds of small code puzzles. But instead of just being unrelated microcontent, each puzzle is a tiny stimulus that

teaches a coding concept or language feature. Hence, each puzzle pushes your knowledge *in the same direction*.

Active Beats Passive Learning: Robust scientific evidence shows that active learning improves students' learning performance. A popular study shows that one of the best active learning techniques is *practice testing*.¹ In this learning technique, you test your knowledge even if you have not learned everything yet. The study argues that students must feel safe during these tests. Therefore, the tests must be low-stake, i.e., students have little to lose. After the test, students get feedback about the correctness of the tests. The study shows that practice testing boosts long-term retention of the material by almost a factor of 10. As it turns out, solving a daily code puzzle is not just another learning technique—it is one of the best.

Although active learning is twice as effective, most books focus on passive learning. The author delivers information; the student passively consumes it. Puzzle-based learning with this book fixes this mismatch between research and everyday practice. In contrast to other books, this book makes active learning a first-class citizen. Solving code puzzles is an in-

¹<http://journals.sagepub.com/doi/abs/10.1177/1529100612453266>

herent active learning technique. You must develop the solution yourself, in every single puzzle.

Make Code a First-class Citizen: Each grandmaster of chess has spent tens of thousands of hours looking into a nearly infinite number of chess positions. Over time, they develop a powerful skill: the intuition of the expert. When presented with a new position, they can name a small number of strong candidate moves within seconds. For ordinary people, the position of a single chess piece is one chunk of information. Hence they can only memorize the position of about six chess pieces. But chess grandmasters view a whole position or a sequence of moves as a single chunk of information. The extensive training and experience have burned strong patterns into their biological neural networks. Their brain can hold much more information—a result of the good learning environment they have put themselves in. Chess grandmasters learn by practicing isolated stimuli again and again until they have mastered them. Then they move on to more complex stimuli.

Puzzle-based learning is code-centric: code is the isolated stimuli. You will find yourself staring at the code for a long time until the insight strikes. This creates new synapses in your brain that help you understand, write, and read code fast. Placing code in

the center of the whole learning process creates an environment in which you will develop the powerful intuition of the expert. *Maximize the learning time you spend looking at code rather than at other stimuli.*

In the following chapters, we will show you how to create your own regular expressions patterns using Python's regex library `re`. You'll need 20 minutes or so to read each chapter. After each chapter, you'll have the opportunity to watch the related regex video on YouTube and solve the practical code puzzle (interactive or in the book). We'll give you two links for the video and the puzzle which you can use for interactive learning.

— 5 —

Basics

Are you motivated to build your regex superpower? If you are not until now, please revisit Chapter 2 to recall the broad range of applications that thrive on top of regular expressions. It's highly likely that one day you will find yourself a professional in one of those application domains and your knowledge of regular expressions will definitely make a difference in how well you fare.

Now, for this chapter, let's start slowly with the very basics of regular expressions.

5.1 What’s a Regex Pattern?

The English language consists of a set of words that are grammatical. All other words are non-grammatical. Both sets are infinitely large for all practical matters. For example, the word *programming* is grammatically correct while the word *qwertzuiopü* is not. For more than one hundred years, researchers have tried to find rules that describe whether a word is grammatical or ungrammatical.

Think of a regular expression as describing a language, too. If you create a regex pattern, you’ve just created your own language—a fascinating thought, isn’t it? In fact, regular expressions and *formal language theory* are closely related fields¹.

You use regular expressions to define the set of “grammatical” words—grammatical for the little language you’re designing. How do you do this? You define a set of one or more words through regex patterns (see Figure 5.1). The pattern can be a simple string

¹For instance, mathematician Stephen Cole Kleene developed regular expressions in the 1950s as a means to create *regular languages*. But don’t worry, this is as far as we’ll get into formal language theory. Well, if you read this whole book, you’ll understand the basics of formal language theory—although you won’t realize it if you don’t decide to pursue a computer science degree

	Regex pattern	
	<code>[\w\.\.]*\w+\(.*?\)</code>	
<code>re.match(a, b, c)</code>		Hello world
<code>match(a, b, c)</code>		match (
<code>re.re.exec()</code>		(noop)
Matching words		Non-matching words

Figure 5.1: Each regex pattern defines a set of grammatical and a set of ungrammatical words. You call them *matching* and *non-matching words*.

of characters like 'alice' or a complicated string of special symbols, quantifiers, and meta-characters like '`[\w\.\.]*\w+\(.*?\)`'. The first pattern defines a single word (the word 'alice') while the second pattern defines an infinite number of words (because of the infinite combinations of characters it is a match). We used the second pattern—that describes valid Python method calls with their arguments—to style Python method calls with a special code font in this book, which is yet another application of regular expres-

sions!

Of course, we don't expect that you already understand what the complicated pattern does and how it works. But just so you know: if you've read this book, understanding it will be a breeze for you!

Many teachers shy away from giving the regex beginner too much technical detail about how the regex processing internally works. But we believe it'll help you make regular expressions more accessible. Figure 5.2 shows the process of matching the pattern 'hello' in the string 'hello world'. The *regex engine* internally goes over the string and keeps track of partial matches that have occurred so far. As soon as the match is done, the engine moves on to either further matching the pattern in the remaining string or it terminates if there's no further match possible.

The regex pattern defines a whole language of valid words. To put it differently, all words in your defined language *match* your pattern. The regex engine is responsible for finding occurrences of the words in your language—it finds the *matches* of the pattern in a given text.

As you've seen in the applications chapter, finding those matches is very useful. For instance, you can extract interesting parts of the text (e.g., the Python

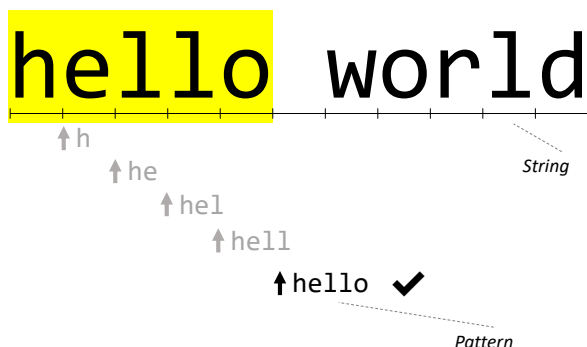


Figure 5.2: Matching the simple pattern 'hello' in the string 'hello world'.

method calls in this book), or even substitute certain parts of the text (e.g., translating Python method calls to, say, C++ method calls).

Throughout this book, we will use the methods of Python's regex library `re`, as a way of practically familiarizing you with regular expressions. From this point on in the book, you'll need a quick introduction to the single most important method of the `re` library. Don't worry, we describe this method here and now.

5.2 The Method to Find All Matches

The `re.findall()` method is the most basic way of using regular expression patterns in Python.

How Does It Work?

The `re.findall(pattern, string, flags=0)` method finds all occurrences of the `pattern` in the `string` and returns a list of all matching substrings. It has up to three arguments.

- **pattern:** the regular expression pattern that you want to match.
- **string:** the string in which you want to search the pattern.
- **flags** (optional argument): a more advanced modifier that allows you to customize the behavior of the function. You can forget about this argument for now. We'll come back to it later in the book.

Return Value: The `re.findall()` method returns a list of strings. Each string element is a matching substring of the string argument. You say, the substring *matches* your `pattern`.

5.2. THE METHOD TO FIND ALL MATCHES 41

Let's check out a few examples.

You import the `re` module and create the text string to be searched for the regex patterns:

```
import re

# Shakespeare
text = '''
    Ha! let me see her: out, alas! he's cold:
    Her blood is settled, and her joints are stiff;
    Life and these lips have long been separated:
    Death lies on her like an untimely frost
    Upon the sweetest flower of all the field.
'''
```

Let's say, you want to search the text for the string `'her'`:

```
>>> re.findall('her', text)
['her', 'her', 'her']
```

The first argument is the pattern. In your case, it's the string `'her'`. The second argument is the text to be analyzed stored as a multi-line string in the variable `text`.

5.3 Summary

- A pattern defines one or more words.
- The set of words defined by your pattern is your personal language.
- If you go through all the trouble creating your personal language, you'd love to see how famous writers such as Shakespeare used *your* words in their texts, right?
- If one of your words appear in the text, you've found a *match*!
- The `re.findall(pattern, text)` method gives you all occurrences of the found pattern in a list of strings.

Video

Watch the short chapter video on YouTube:



https://youtu.be/Lj_QGc7zWUA

Puzzle

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



[https://app.finxter.com/learn/
computer/science/565](https://app.finxter.com/learn/computer/science/565)

Special Symbols

Regular expressions are built from characters. There are two types of characters: *literal characters* and *special characters*.

6.1 Literal Characters

Let's start with what you already know: you write regular expressions to match a given pattern in a given string.

Note: We use the term *regular expression* when we talk about the larger field and the term *pattern* when we talk about the concrete matching process. But this is only a preference because these terms are really

interchangeable. A pattern is a regular expression. And a regular expression is a pattern. For example, some coders use the term *pattern matching* when they talk about regular expressions.

In its most basic form, a pattern is a literal character. For example, if you search for the regex pattern 'd' in the string 'hello world', it is matched in the last position. Similarly, a regex pattern with a combination of literal characters, say 'ld', is valid and matches the last two characters in 'hello world'.

Special Characters

The power of regular expressions comes from their abstraction capability. Like you need air to breathe, regular expressions need special symbols for their construction. For instance, instead of writing the character set [abcdefghijklmnopqrstuvwxyz], you'd write [a-z] (see Chapter 7) or even \w (as we will discuss in detail in this chapter). The latter is a special regex character—and every regex master uses it often. In fact, regex experts seldom match literal characters. They often use more advanced constructs or special characters for brevity, expressiveness, or generality.

So what are the special characters you can use in

your regex patterns?

Well, we will introduce them one-by-one now. Note that many of these characters are also available in other regex-supporting languages such as Perl. Thus, if you study the following list carefully, you'll undoubtedly improve your conceptual clarity in writing your own regular expressions—independent of the concrete coding-language you use.

- `\n` The newline character is not a special character particular to regular expressions. It's one of the most widely-used, standard characters. However, leaving it out here would definitely leave our list of special characters incomplete. For example, the regex `'hello\nworld'` matches a string where the string `'hello'` is placed in one line and the string `'world'` is placed into the second line.
- `\t` Just like the newline character, the tabular character is also not a “regex-specific” character. It just encodes the tabular space `' '` which is different to a sequence of whitespaces (even if it doesn't look different here). For example, the regex `'hello\n\tworld'` matches the string that consists of `'hello'` in the first line and `' world'` in the second line (with a leading tab character).

- `\s` Unlike the newline and the tabular character, the whitespace character is a special symbol of the regex library. You'll find it in many other programming languages, too. It solves an important problem: you often don't know which type of whitespace is used—tabular characters, simple whitespaces, or even newlines. The whitespace character `'\s'` matches any of them. For example, the regex `'\s*hello\s+world'` matches the string `' \t \n hello \n \n \t world'`, as well as `'hello world'`.
- `\S` The whitespace-negation character (with capital 'S') matches everything that does not match `\s`.
- `\w` The word character regex simplifies text processing significantly. It represents the class of all characters used in typical words (A-Z, a-z, 0-9, and '_'). This simplifies the writing of complex regular expressions significantly. For example, the regex `'\w+'` matches the strings `'hello'`, `'bye'`, `'Python'`, and `'Python_is_great'`.
- `\W` The word-character-negation (note the capital 'W'). It matches any character that is not a word character.

- \b The word boundary is also a special symbol used in many regex tools. You can use it to match, as the name suggests, the boundary between the a word character (\w) and a non-word (\W) character. But note that it matches only the empty string! You may ask: why does it exist if it doesn't match any character? The reason is that it doesn't "consume" the character right in front or right after a word. This way, you can search for whole words (or parts of words) and return only the word but not the delimiting characters that separate the words.
- \d The digit character matches all numeric symbols between 0 and 9. Use it to match integers with an arbitrary number of digits: the regex '\d+' matches integer numbers '10', '1000', '942', and '99999999999'.
- \D Matches any non-digit character. This is the inverse of \d and it's equivalent to [^0-9] (see Chapter 7).

These are the most important special symbols and characters. But before you move on to the next chapter, let's try to understand them better by studying some examples!

```
import re

text = '''
    Ha! let me see her: out, alas! he's cold:
    Her blood is settled, and her joints are stiff;
    Life and these lips have long been separated:
    Death lies on her like an untimely frost
    Upon the sweetest flower of all the field.
'''

print(re.findall('\w+\W+\w+', text))
```

Matches each pair of words in the text. Here's the output:

```
['Ha! let', 'me see', 'her: out', 'alas! he',
's cold', 'Her blood', 'is settled', 'and her',
'joints are', 'stiff;\n Life', 'and these',
'lips have', 'long been', 'separated:\n Death',
'lies on', 'her like', 'an untimely',
'frost\n Upon', 'the sweetest', 'flower of', 'all
↪ the']
```

Note that it matches also across new lines: 'stiff;\n Life' also matches!

Note also that matched text is “consumed” and doesn't match again. That's why the combination 'let me' is not a matching substring.

```
print(re.findall('\d', text))
```

No integers in the text:

```
[]
```

The output is empty. What if you search for all occurrences of a newline character, followed by a tab character?

```
print(re.findall('\n\t', text))
```

Match all occurrences where a tab follows a newline:

```
[]
```

No match because each line starts with a sequence of four whitespaces rather than the tab character.

```
print(re.findall('\n ', text))
```

Next, you're going to match all occurrences where 4 whitespaces ' ' follow a newline:

```
['\n ', '\n ', '\n ', '\n ', '\n ']
```

Yes! It matches all five lines.

6.2 Summary

- Regular expressions are built from characters.
- An important class of characters is the class of literal characters. In principle, you can use all Unicode literal characters in your regex pattern.
- Another class of characters is the class of special characters. They make your life much easier, and your patterns more readable.

Video

Watch the short chapter video on YouTube:



<https://youtu.be/hSy0xea-8p8>

Puzzle

```
import re

# Shakespeare
text = '!!!'
```

```
Ha! let me see her: out, alas! he's cold:
Her blood is settled, and her joints are stiff;
Life and these lips have long been separated:
Death lies on her like an untimely frost
Upon the sweetest flower of all the field.
'''

print(len(re.findall('\s\w\w\s', text)))
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



<https://app.finxter.com/learn/computer/science/577>

Character Sets

There are many terms describing the same powerful concept: “character class”, “character range”, or “character group”. However, the most precise term is “character set” as introduced in the official Python regex docs. So in this book, we’ll use this term throughout.

7.1 What’s a Character Set?

The best way to describe a character set is, unsurprisingly, as a set of characters. If you use a character set in a regular expression pattern, you tell the regex engine to look for an arbitrary character from that set.

As you may know, a set is an *unordered collection of unique elements*. So each character in a character

set is unique, and the order doesn't matter (with a few minor exceptions).

Here's an example of a character set as used in a regular expression:

```
>>> import re
>>> re.findall('[abcde]', 'hello world!')
['e', 'd']
```

You use the `re.findall(pattern, string)` method to match the pattern `'[abcde]'` in the string `'hello world!'`. You can think of all characters `a`, `b`, `c`, `d`, and `e` as being in an OR relation: either of them is a valid match. This OR relation between the characters is represented by the square brackets around them (`[]`)

The regex engine moves from left to right, scanning over the string `'hello world!'` and simultaneously trying to match the character set. Two characters from the text `'hello world!'` are in the character set—they are valid matches and returned by the `re.findall()` method.

You can simplify many character sets by using the range symbol `'-'` that has a special meaning within square brackets: `[a-z]` reads “match any character from

a to z”, while `[0-9]` reads “match any character from 0 to 9”.

Here’s the previous example, simplified:

```
>>> re.findall('[a-e]', 'hello world!')
['e', 'd']
```

You can even combine multiple character ranges in a single character set:

```
>>> re.findall('[a-zA-E0-4]', 'hello WORLD 42!')
['e', 'D', '4', '2']
```

Here, you match three ranges: lowercase characters from 'a' to 'e', uppercase characters from 'A' to 'E', and numbers from '0' to '4'.

Note that the ranges are inclusive, *so both start and stop symbols are included in the range.*

7.2 Negative Character Set

But what if you want to match all characters—except some? You can achieve this with a negative character set. The negative character set works just like a (positive) character set, but with one difference: it matches all characters that are *not* in the character

set. It is represented by an additional `^` character inside the square brackets.

Here's an example where you match all sequences of characters that do not contain characters 'a', 'b', 'c', 'd', or 'e':

```
>>> import re
>>> re.findall('[^a-e]+', 'hello world')
['h', 'llo worl']
```

Note the additional use of the “at-least-once quantifier +” in this example that matches at least one occurrence of the preceding regex (see Chapter 10). It is not necessary that you use it for negative sets. Its just there match more than a single character (i.e., sequences).

There are only two such sequences: the one-character sequence 'h' and the eight-character sequence 'llo worl'. You can see that even the space character matches the negative character set.

7.3 Summary

- If you use a character set `[XYZ]` in a regex pattern, the regex engine chooses an arbitrary character from the set: X, Y, or Z.

- The characters in a set have an *either-or* relation. Any of them is an acceptable match.
- The negative character set `[^...]` matches all characters that are not enclosed in the square brackets.

Video

Watch the short chapter video on YouTube:



<https://youtu.be/lrI5wmZo-mY>

Puzzle

```
import re

def special_name(s):
    return True if re.match('j[a-z]+n', s) else False

print(special_name('chris'))
print(special_name('johanna'))
print(special_name('joan'))
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



<https://app.finxter.com/learn/computer/science/570>

If you need help with the `re.match()` method, check out the following mini-tutorial.

7.4 Intermezzo: Match Function

The `re.match(pattern, string)` method matches the pattern at the beginning of the string and returns a match object. Here's the specification:

```
re.match(pattern, string, flags=0)
```

The `re.match()` method has up to three arguments.

- **pattern**: the regular expression pattern that you want to match.
- **string**: the string in which you want to search the pattern.
- **flags** (optional argument): a more advanced modifier that allows you to customize the behavior of the function.

Return Value: The `re.match()` method returns a match object. You can access the start and end positions of the match in the string by calling the methods `m.start` and `m.end()` on the match object `m`:

```
>>> m = re.match('h...o', 'hello world')
>>> m.start()
0
>>> m.end()
5
>>> 'hello world'[m.start():m.end()]
'hello'
```

In the first line, you create a match object `m` by using the `re.match()` method. The pattern `'h...o'` matches in the string `'hello world'` at start position

0. You use the start and end position to access the substring that matches the pattern (using the popular Python technique of slicing). But note that as the `match()` method always attempts to match only at the beginning of the string, the `m.start()` method will always return zero.

Let's check out a few examples of `re.match()`!

First, you import the `re` module and create the text string to be searched for the regex patterns:

```
>>> import re
>>> text = '''
    Ha! let me see her: out, alas! he's cold:
    Her blood is settled, and her joints are stiff;
    Life and these lips have long been separated:
    Death lies on her like an untimely frost
    Upon the sweetest flower of all the field.
    '''
```

Let's say you want to search the text for the string 'her':

```
>>> re.match('lips', text)
>>>
```

The first argument is the pattern to be found: the string `'lips'`. The second argument is the text to be

analyzed. You stored the multi-line string in the variable `text`—so you take this as the second argument. The third argument `flags` of the `match()` method is optional.

There's no output: The `re.match()` method did not return a match object. Why? Because at the beginning of the string, there's no match for the regex pattern `'lips'`. You can fix this by matching all the characters that precede the string `'lips'` in the text:

```
>>> re.match('(.|\n)*lips', text)
<re.Match object; span=(0, 122), match="\n    Ha! let
me see her: out, alas! he's cold:\n">
```

The regex `(.|\n)*lips` matches an arbitrary prefix followed by the string `'lips'`. This results in a new match object that matches a huge substring from position 0 to position 122. Note that the match object doesn't print the whole substring to the shell. If you access the matched substring, you'll get the following result:

```
>>> m = re.match('(.|\n)*lips', text)
>>> text[m.start():m.end()]
"\n    Ha! let me see her: out, alas! he's cold:\n
Her blood is settled, and her joints are stiff;\n
Life and these lips"
```

Interestingly, you can also achieve the same thing by specifying the third flag argument as follows:

```
>>> m = re.match('.*lips', text, flags=re.DOTALL)
>>> text[m.start():m.end()]
"\n    Ha! let me see her: out, alas! he's cold:\n
↳ Her blood is settled, and her joints are stiff;\n
↳ Life and these lips"
```

The `re.DOTALL` flag ensures that the dot matches all characters, including the new line character.

7.5 Methods `re.match()` vs `re.findall()`

There are two differences:

- `re.match(pattern, string)` returns a match object while `re.findall(pattern, string)` returns a list of matching strings.
- `re.match(pattern, string)` returns only the first match in the string—and only at the beginning—while `re.findall(pattern, string)` returns all matches in the string.

You can see both in the following example:

```
>>> text = 'Python is superior to Python'
>>> re.match('Py...n', text)
<re.Match object; span=(0, 6), match='Python'>
>>> re.findall('Py...n', text)
['Python', 'Python']
```

The string 'Python is superior to Python' contains two occurrences of 'Python'. The `match()` method only returns a match object of the first occurrence. The `findall()` method returns a list of all occurrences.

Let's shortly discuss the `re.fullmatch(pattern, string)` method which is a variant of the `re.match(pattern, string)` method.

7.6 `re.fullmatch()`

The `re.fullmatch(pattern, string)` method returns a match object if the pattern matches the whole string. It has up to three arguments.

- `pattern`: the regular expression pattern that you want to match.

- **string**: the string in which you want to search the pattern.
- **flags** (optional argument): a more advanced modifier that allows you to customize the behavior of the function.

Return Value: The `re.fullmatch()` method returns a match object. Let's check out a few examples of `re.fullmatch()`!

First, you import the `re` module and create the text string to be searched for the regex patterns:

```
>>> import re
>>> text = '''
Call me Ishmael. Some years ago--never mind how long
↳ precisely
--having little or no money in my purse, and nothing
↳ particular
to interest me on shore, I thought I would sail about
↳ a little
and see the watery part of the world.
'''
```

Let's say you want to match the full text with this regular expression:

```
>>> re.fullmatch('Call(.\n)*', text)
>>>
```

The first argument is the pattern to be found: 'Call(.\n)*'. The second argument is the text to be analyzed. You stored the multi-line string in the variable `text`—so you take this as the second argument.

The method did not return a match object. Why? Because at the beginning of the string, there's no match for the 'Call' part of the regex. The regex starts with an empty line! So how to fix this? Simple, by matching a new line character '\n' at the beginning of the string.

```
>>> re.fullmatch('\nCall(.\n)*', text)
<re.Match object; span=(0, 229), match='\nCall me
Ishmael. Some years ago--never mind how>
```

The regex `(.\n)*` matches an arbitrary number of characters (newline characters or not) after the prefix '\nCall'. As the regex matches the whole text, the result is a match object (rather than `None`). Note that there are 229 matching positions, so the string included in the resulting match object is only the prefix

of the whole matching string. Beginner coders often overlook this fact.

7.7 Methods `re.fullmatch()` vs `re.match()`

The methods `re.fullmatch(pattern, string)` and `re.match(pattern, string)` both return a match object. Both attempt to match at the beginning of the string. The only difference is that `re.fullmatch()` also attempts to match the end of the string as well: it wants to match the whole string!

You can see this difference in the following code:

```
>>> text = 'More with less'
>>> re.match('More', text)
<re.Match object; span=(0, 4), match='More'>
>>> re.fullmatch('More', text)
>>>
```

The `re.match('More', text)` method matches the string 'More' at the beginning of the string 'More with less'. But the `re.fullmatch('More', text)` method does not match the whole text. Therefore, it

returns the `None` object—nothing is printed to your shell!

Dot Regex

Now let's learn one of the most frequently used regular expression pattern next: the dot regex.

8.1 What's the Dot Regex?

The dot regex `.` matches all characters except the newline character. For example, the regular expression `'...'` matches strings `'hey'` and `'tom'` because both words consist of three arbitrary, non-newline characters. But it does not match the string `'yo\ntom'` which contains the newline character `'\n'`.

Let's study some basic examples for deeper understanding.

```
>>> import re
>>>
>>> text = '''But then I saw no harm, and then I
↳ heard
Each syllable that breath made up between them.'''
>>> re.findall('B..', text)
['But']
>>> re.findall('heard.Each', text)
[]
>>> re.findall('heard\nEach', text)
['heard\nEach']
>>>
```

First, you import Python's `re` library for regular expression handling. Then, you create a multi-line text using the triple string quotes.

Let's deconstruct this and dive into the first pattern matching attempt:

```
>>> re.findall('B..', text)
['But']
```

The `re.findall(pattern, string)` method finds all occurrences of the pattern in the string and returns a list of all matching substrings. The first argument is the pattern `'B..'`. The second argument is the string to be analyzed. You want to find all patterns starting with the `'B'` character, followed by two arbitrary

characters except the newline character. There's only one such occurrence in the text: 'But'.

The second pattern matching example shows that the dot doesn't match the newline character:

```
>>> re.findall('heard.Each', text)
[]
```

You're looking at the simple pattern 'heard.**Each**'. You want to find all occurrences of string 'heard' followed by an arbitrary non-whitespace character, followed by the string 'Each'.

But such a pattern doesn't exist in the text! Many coders intuitively read the dot regex as an arbitrary character. You must be aware that the dot regex matches an arbitrary character *except the newline character*. Not considering the second part of the definition is a source of many bugs when designing regular expressions.

The third pattern matching example shows how to explicitly match the newline character '**\n**' instead:

```
>>> re.findall('heard\nEach', text)
['heard\nEach']
```


Now, the regex engine successfully matches the substring. Naturally, the following relevant question arises:

8.2 Intermezzo: Flags

In the previous example, you've seen a third argument **flags**. What are they and how do they work?

Flags allow you to control the regular expression engine. Because regular expressions are so powerful, they are a useful way of switching on and off certain features (e.g., whether to ignore capitalization when matching your regex).

For example, here's how the third argument flags is used in the `re.findall()` method:

```
re.findall(pattern, string, flags=0)
```

So the flags argument seems to be an integer argument with the default value of 0. To control the default regex behavior, you simply use one of the predefined integer values. You can access these predefined values via the `re` library:

- `re.ASCII`: If you don't use this flag, the special Python regex symbols `\w`, `\W`, `\b`, `\B`, `\d`, `\D`, `\s` and `\S` will match Unicode characters. If you use this flag, those special symbols will match only ASCII characters — as the name suggests.
- `re.A`: Same as `re.ASCII`.
- `re.DEBUG`: If you use this flag, Python will print some useful information to the shell that helps you debugging your regex.
- `re.IGNORECASE`: If you use this flag, the regex engine will perform case-insensitive matching. So if you're searching for `[A-Z]`, it will also match `[a-z]`.
- `re.I`: Same as `re.IGNORECASE`.
- `re.LOCALE`: Don't use this flag — ever. It's deprecated—the idea was to perform case-insensitive matching depending on your current locale. But it isn't reliable.
- `re.L`: Same as `re.LOCALE`.
- `re.MULTILINE`: This flag switches on the following feature: the start-of-the-string regex `^`

matches at the beginning of each line (rather than only at the beginning of the string). The same holds for the end-of-the-string regex '\$' that now matches also at the end of each line in a multi-line string.

- **re.M:** Same as **re.MULTILINE**
- **re.DOTALL:** Without using this flag, the dot regex '.' matches all characters except the newline character '\n'. Switch on this flag to really match all characters including the newline character.
- **re.S:** Same as **re.DOTALL**.
- **re.VERBOSE:** To improve the readability of complicated regular expressions, you may want to allow comments and (multi-line) formatting of the regex itself. This is possible with this flag: all whitespace characters and lines that start with the character '#' are ignored in the regex.
- **re.X:** Same as **re.VERBOSE**

To use these flags, simply include the flag as the optional flag argument as follows:

```
import re

text = '''
    Ha! let me see her: out, alas! he's cold:
    Her blood is settled, and her joints are stiff;
    Life and these lips have long been separated:
    Death lies on her like an untimely frost
    Upon the sweetest flower of all the field.
'''

print(re.findall('HER', text, flags=re.IGNORECASE))
# ['her', 'Her', 'her', 'her']
```

As you see, the `re.IGNORECASE` flag ensures that all occurrences of the string `'her'` match the pattern `'HER'`—no matter their capitalization. You can also use multiple flags by simply adding them together (sum them up) as follows:

```
import re

text = '''
    Ha! let me see her: out, alas! he's cold:
    Her blood is settled, and her joints are stiff;
    Life and these lips have long been separated:
    Death lies on her like an untimely frost
    Upon the sweetest flower of all the field.
'''
```

```
print(re.findall('  HER  # Ignored', text,  
               flags=re.IGNORECASE + re.VERBOSE))  
# ['her', 'Her', 'her', 'her']
```

You use both flags `re.IGNORECASE` (all occurrences of lower- or uppercase string variants of `'her'` are matched) and `re.VERBOSE` (ignore comments and whitespaces in the regex). You sum them together `re.IGNORECASE + re.VERBOSE` to indicate that you want to take both.

8.3 Match Newline

The dot regex `.` matches a single arbitrary character—except the newline character. But what if you do want to match the newline character, too? There are two main ways to accomplish this.

- Use the `re.DOTALL` flag.
- Use a character set `[\n]`.

Here's the concrete example showing both cases:

```
>>> import re  
>>>
```

```
>>> s = '''hello
python'''
>>> re.findall('o.p', s)
[]
>>> re.findall('o.p', s, flags=re.DOTALL)
['o\np']
>>> re.findall('o[.\n]p', s)
['o\np']
```

You want to find the regex pattern 'o.p' in the multi-line string. But the dot regex doesn't match the newline character per default. Only if you set the flag `re.DOTALL`, the newline character will also be considered a valid match by the regex engine.

An alternative is to use the slightly more complicated regex pattern `[.\n]`. The square brackets enclose a character set—a set of characters that are all a valid match (see Chapter 7). Think of a character set as an OR operation: exactly one character must match.

8.4 Match the Dot Character

If you use the character '.' in a regular expression, Python assumes that it's the dot regex you're talking about. But what if you actually want to match

the period character—for example, at the end of a sentence?

In this case, simply escape the dot regex with the backslash: `'\.'`. The backslash nullifies the meaning of the special symbol `'.'` in the regex. The regex engine now knows that you're looking for the period character, not an arbitrary character except a new-line.

Here's an example:

```
>>> import re
>>> text = 'Python. Is. Great. Period.'
>>> re.findall('\.', text)
['.', '.', '.', '.']
```

The `findall()` method returns all four periods in the sentence as matching substrings for the regex `'\.'`. In this example, you'll learn how to combine it with other regular expressions:

```
>>> re.findall('\.\s', text)
['. ', '. ', '. ']
```

Now, you're looking for a period character followed by arbitrary whitespace. There are only three such matching substrings in the text.

In the next example, you'll learn how to combine this with a character set:

```
>>> re.findall('[st]\.', text)
['s.', 't.']
```

You want to find either character 's' or character 't' followed by the period character '.'. Two substrings match this regex.

Note that skipping the backslash is required. If you forget this, it can lead to strange behavior:

```
>>> re.findall('[st].', text)
['th', 's.', 't.']
```

As an arbitrary character is allowed after the character set, the substring 'th' also matches the regex.

8.5 Summary

You've learned everything you need to know about the dot regex `.` in this tutorial.

- The dot regex matches all characters except the newline character. For example, the regular expression `'...'` matches strings `'hey'` and

'tom'. But it does not match the string 'yo\ntom' which contains the newline character '\n'.

- If you want to match the period character, just escape the dot '\.' to get rid of its special meaning.

Video

Watch the short chapter video on YouTube:



<https://youtu.be/UVytEt4ps6w>

Puzzle

```
import re

text = 'Speedy Gonzalez'
matches = re.findall('.', text)
result = len(matches) == len(text)

print(result)
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



<https://app.finxter.com/learn/computer/science/546>

Asterisk Quantifier

Every computer scientist knows the *asterisk quantifier* of regular expressions. But many non-techies know it, too. For example, if you search for a text file `*.txt` on your computer, you've just used the asterisk quantifier.

9.1 What's the Asterisk Quantifier?

Given is a regex pattern `A`. The pattern combined with the asterisk quantifier `A*` matches zero or more occurrences of `A`. The asterisk quantifier `*` always applies to the preceding regular expression. For example, the regular expression `'yes*'` matches strings `'ye'`,

'yes', and 'yessssssss'. But it does not match the empty string in this specific case. Why? Because the asterisk quantifier `*` does not apply to the whole regex 'yes' but only to the preceding regex 's'.

9.2 Asterisk Example

Let's study two basic examples to help you gain a deeper understanding.

```
>>> import re
>>> text = 'finxter for fast and fun python learning'
>>> re.findall('f.* ', text)
['finxter for fast and fun python ']
>>> re.findall('f[a-z]*', text)
['finxter', 'for', 'fast', 'fun']
```

Don't worry if you had problems understanding those examples. You'll learn about them next. Here's the first example:

```
>>> re.findall('f.* ', text)
['finxter for fast and fun python ']
```

The pattern `'f.* '` matches all substrings that start with the character `'f'`, followed by an arbitrary number of optional characters, and followed by a space character.

The `findall()` method returns only one matching substring: `'finxter for fast and fun python'`. Why is that? The asterisk quantifier is greedy—it tries to match as many occurrences of the preceding regex as possible. Thus, it wants to match as many arbitrary characters as possible (well, it ensures that the pattern still matches but that's the only restriction). The whole sentence a valid match. Therefore, the regex engine `'consumes'` the whole sentence.

The second example is the following:

```
>>> re.findall('f[a-z]*', text)
['finxter', 'for', 'fast', 'fun']
```

This regex achieves almost the same thing: finding all words starting with `f`. But you use the asterisk quantifier in combination with a character set that defines which characters are valid matches.

As you already know, you can use character ranges in the character set. For example, the character range `[a-z]` matches one lowercase character in the alphabet, while the character range `[A-Z]` matches one uppercase character in the alphabet. Note that the empty space is not part of the character set, so it won't be matched if it appears in the text. Thus, the

result is the same list of words that start with character `f`: `'finxter '`, `'for '`, `'fast '`, and `'fun '`.

9.3 Match the Asterisk Character

What if you search for the asterisk (or star) character itself? How can you search for it in a string?

The answer is simple: escape the asterisk character in your regular expression using the backslash. In particular, use `'\*'` instead of `'*'`. Here's an example:

```
>>> import re
>>> text = 'Python is ***great***'
>>> re.findall('\*', text)
['*', '*', '*', '*', '*', '*']
>>> re.findall('\**', text)
['', '', '', '', '', '', '', '', '', '', '', '***', '',
↪  '', '', '', '', '***', '']
>>> re.findall('\*+', text)
['***', '***']
```

You find all occurrences of the star symbol in the text by using the regex `'\*'`. Consequently, if you use the regex `'**'`, you search for an arbitrary number of occurrences of the asterisk symbol (including zero occurrences). And if you would like to search

for all maximal number of occurrences of subsequent asterisk symbols in a text, you'd use the regex '`\**`' (see Chapter 10).

9.4 Summary

- The asterisk quantifier applies to its preceding pattern.
- It matches the preceding pattern zero or multiple times.
- It's greedy.
- Match the star symbol by escaping the asterisk quantifier `\*`.

Video

Watch the short chapter video on YouTube:



<https://youtu.be/KPcDBzw6IX0>

Puzzle

```
import re

famous_tweet = '''
I'm going give someone random who retweets this
tweet $10,000 because it's my birthday and I feel
like being nice (you have to be following me so
I can dm you the code if you win)
@MrBeastYT
#Twitter
'''

user = re.findall('@.*', famous_tweet)
print(user[0])
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



<https://app.finxter.com/learn/computer/science/566>

Plus Quantifier

Let's learn about the second major regex quantifier: the + symbol.

10.1 What's the Plus Quantifier?

Say, you have any regular expression **A**. The pattern **A+** matches *one* or more occurrences of **A**. In contrast, the asterisk quantifier matches zero or more occurrences of **A**.

You call the + symbol the *at-least-once quantifier* because it requires at least one occurrence of the preceding pattern. For example, the regular expression **'yes+'** matches strings **'yes'**, **'yess'**, and **'yesssssss'**. But it does neither match the string **'ye'**, nor the

empty string '' because the plus quantifier + does not apply to the whole regex 'yes' but only to the preceding pattern 's'. Let's study some examples to help you gain a deeper understanding.

```
>>> import re
>>> re.findall('a+b', 'aaaaaab')
['aaaaaab']
>>> re.findall('ab+', 'aaaaabb')
['abb']
>>> re.findall('ab+', 'aaaaabbbbb')
['abbbbb']
>>> re.findall('ab+?', 'aaaaabbbbb')
['ab']
>>> re.findall('ab+', 'aaaaaa')
[]
>>> re.findall('[a-z]+', 'hello world')
['hello', 'world']
```

Next, we'll explain those examples one by one.

10.2 Plus (+) Quantifiers Examples

Here's the first example:

```
>>> re.findall('a+b', 'aaaaaab')
['aaaaaab']
```

The first argument of the `findall()` method is the regular expression pattern `'a+b'` and the second argument is the analyzed string `'aaaaaab'`. You want to find all patterns in the string that start with at least one, but possibly many, characters `'a'`, followed by the character `'b'`.

There's one matching substring: `'aaaaaab'`. As the plus quantifier `+` is greedy, it matches as many occurrences of the preceding regex as possible. In our case, it wants to match as many arbitrary characters as possible. Again, the regex engine “consumes” the whole sentence.

The second example is similar:

```
>>> re.findall('ab+', 'aaaaabb')
['abb']
```

You search for the character `'a'` followed by at least one character `'b'`. As the plus (+) quantifier is greedy, it matches as many `'b'`s as it can lay its hands on.

For the sake of your thorough understanding, let's have a look at the other given example:

```
>>> re.findall('ab+', 'aaaaaa')
[]
```

You can see that the plus (+) quantifier requires to match at least one occurrence of the preceding regex. However, the character 'b' cannot be matched. So, the result is the empty list indicating that no matching substring was found.

Another interesting example is the following:

```
>>> re.findall('[a-z]+', 'hello world')
['hello', 'world']
```

You use the plus (+) quantifier in combination with a character set that defines which characters are valid matches.

10.3 Match the Plus (+) Symbol

You know that the plus quantifier matches at least one of the preceding regular expression. But what if you search for the plus (+) symbol itself? If you've studied the previous chapter about the asterisk quantifier carefully, you can already guess: escape the plus symbol with the backslash. In particular, use '\+' instead of '+'. Here's an example:

```
>>> import re
>>> text = '2 + 2 = 4'
```

```
>>> re.findall(' + ', text)
[]
>>> re.findall(' \+ ', text)
[' + ']
>>> re.findall(' \++ ', '2 ++++ 2 = 4')
[' ++++ ']
```

If you want to find the '+' symbol in your string, you need to escape it by using the backslash. If you don't do this, the Python regex engine will interpret it as a regular “at-least-once regex”. Of course, you can combine the escaped plus symbol '\+' with the “at-least-once” regex searching for at least one occurrences of the plus symbol.

10.4 Summary

- Regex **A+** matches one or more occurrences of regex **A**.
- The **+** symbol is the at-least-once quantifier because it requires at least one occurrence of the preceding regex.
- Big news: you can match the plus symbol itself by escaping it like this: **\+**.

Video

Watch the short chapter video on YouTube:



<https://youtu.be/DwJJGvNK1pk>

Puzzle

```
import re

website = '''
<!doctype html>
<html>
  <head>
    <meta charset="utf-8">
  </head>
  <body>
    hello world
  </body>
</html>
'''

print(len(re.findall('<.+>', website)))
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



[https://app.finxter.com/learn/
computer/science/567](https://app.finxter.com/learn/computer/science/567)

Question Mark Quantifier

Congratulations, you're about to learn one of the most frequently used regex operators: the question mark quantifier `A?`. It's the third and final of the major regex quantifiers (the first being the asterisk quantifier and the second the plus quantifier).

11.1 What's the Question Mark Quantifier?

When applied to pattern `A`, the question mark quantifier `A?` matches either zero or one occurrences of the preceding pattern `A`. For example, the regular expression `'hey?'` matches both strings `'he'` and `'hey'` but not the empty string `''`.

Can you figure out the following examples already?

```
>>> import re
>>>
>>> re.findall('aa[cde]?', 'aacde aa aadcde')
['aac', 'aa', 'aad']
>>>
>>> re.findall('aa?', 'accccacccac')
['a', 'a', 'a']
>>>
>>> re.findall('[cd]?[cde]?', 'ccc dd ee')
['cc', 'c', '', 'dd', '', 'e', 'e', '']
```

Let's dive into these examples. Here's the first one:

```
>>> re.findall('aa[cde]?', 'aacde aa aadcde')
['aac', 'aa', 'aad']
```

You want to find all patterns that start with two 'a' characters, followed by one optional character—which can be either 'c', 'd', or 'e'. The `findall()` method returns three matching substrings:

- First, string 'aac' matches the pattern. After Python consumes the matched substring, the remaining substring is 'de aa aadcde'.
- Second, string 'aa' matches the pattern. Python consumes it, which leads to the remaining substring ' aadcde'.

- Third, string 'aad' matches the pattern in the remaining substring. What remains is 'cde', which doesn't contain a matching substring anymore.

The second example is the following:

```
>>> re.findall('aa?', 'acccacccac')
['a', 'a', 'a']
```

In this example, you're looking at the simple pattern 'aa?'. You want to find all occurrences of character 'a' followed by an optional second 'a'. But be aware that the optional second 'a' is not needed for the pattern to match. Therefore, the regex engine finds three matches: the characters 'a'.

The third example is the following:

```
>>> re.findall('[cd]?[cde]?', 'ccc dd ee')
['cc', 'c', '', 'dd', '', 'e', 'e', '']
```

This regex pattern looks complicated: '[cd]?[cde]?'. But is it? Let's break it down step-by-step:

The first part of the regex [cd]? defines a character set [cd] which reads as “match either c or d”. The

question mark quantifier indicates that you want to match either one or zero occurrences of this pattern.

The second part of the regex `[cde]?` defines a character set `[cde]` which reads as “match either c, d, or e”. Again, the question mark indicates the zero-or-one matching requirement. As both parts are optional, the empty string matches the regex pattern. However, the Python regex engine attempts as much as possible.

Thus, the regex engine performs the following steps:

- The first match in the string `'ccc dd ee'` is `'cc'`. The regex engine consumes the matched substring, so the string `'c dd ee'` remains.
- The second match in the remaining string is the character `'c'`. The empty space `' '` does not match the regex so the second part of the regex `[cde]` does not match. Because of the question mark quantifier, this is okay for the regex engine. The remaining string is `' dd ee'`.
- The third match is the empty string `''`. Of course, Python does not attempt to match the same position twice. Thus, it moves on to process the remaining string `'dd ee'`.

- The fourth match is the string 'dd'. The remaining string is ' ee'.
- The fifth match is the string ''. The remaining string is 'ee'.
- The sixth match is the string 'e'. The remaining string is 'e'.
- The seventh match is the string 'e'. The remaining string is ''.
- The eighth match is the string ''. Nothing remains.

This was the most complicated of our examples. Congratulations if you understood it completely!

11.2 Summary

You've learned everything you need to know about the question mark quantifier `?` in this regex tutorial.

- When applied to regular expression `A`, Python's `A?` quantifier matches either zero or one occurrences of `A`.

- The `?` quantifier always applies only to the preceding regular expression.
- We didn't even tell you but can you figure out how to match the question mark character `'?'` itself?

Video

Watch the short chapter video on YouTube:



<https://youtu.be/wfLEGinN3a4>

Puzzle

To solve this puzzle, you'll first need to learn about the `re.match(pattern, string)` method that checks if a **string prefix** matches the regex **pattern** and returns a match object. The match object is just that: an object that stores meta-information about the match, such as the matching position and the matched substring.

In contrast to the `re.findall()` method, the `re.match()` method finds only one match, not all of them. Also in contrast to the `re.findall()` method, it looks only at the beginning of the string. If there's no immediate match at the beginning of a string, it doesn't look further. If there is no matching prefix, the method returns `None`.

Equipped with this information (and some basic Python skills), you may already be able to solve the following advanced puzzle:

```
import re

fruits = [
    "apple",
    "banana",
    "orange",
    "kiwi"
]

lx = lambda x: re.match("?.a", x)
result = list(filter(lx, fruits))

print(len(result))
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



[https://app.finxter.com/learn/
computer/science/552](https://app.finxter.com/learn/computer/science/552)

Quantifier Differentiation

Let’s recap the three quantifiers to nail down their differences.

- The Python Re **A*** quantifier reads as the *zero-or-more regex* (We know it sounds a bit clunky): the preceding regex **A** is matched an arbitrary number of times, including the not-at-all case.
- The Python Re **A+** quantifier reads as the *at-least-once regex*: the preceding regex **A** is matched an arbitrary number of times too—but at least once.
- Another term for the Python Re **A?** quantifier is the **zero-or-one regex**: the preceding regex **A** is matched either zero times or exactly once.

12.1 Asterisk vs Plus

Here's an example that shows the difference:

```
>>> import re
>>> re.findall('ab*', 'aaaaaaaa')
['a', 'a', 'a', 'a', 'a', 'a', 'a', 'a']
>>> re.findall('ab+', 'aaaaaaaa')
[]
```

The regex `'ab*'` matches the character `'a'` in the string, followed by an arbitrary number of occurrences of character `'b'`. The substring `'a'` perfectly matches this formulation. Therefore, you find that the regex matches eight times in the string. The regex `'ab+'` matches the character `'a'`, followed by as many characters `'b'` as possible—but at least one. However, the character `'b'` does not exist so there's no match.

12.2 Asterisk vs Question Mark

Let's have a look at the following example:

```
>>> import re
>>> re.findall('ab?', 'abbbbbbb')
['ab']
>>> re.findall('ab*', 'abbbbbbb')
['abbbbbbb']
```

The regex `'ab?'` matches the character `'a'` in the string, followed by character `'b'` if it exists (which it does in the code). The regex `'ab*'` matches the character `'a'` in the string, followed by as many characters `'b'` as possible.

Question Mark vs Plus

Before you watch the video and solve the puzzle, let's look at the final example in this chapter:

```
>>> import re
>>> re.findall('ab?', 'aaaaaaaa')
['a', 'a', 'a', 'a', 'a', 'a', 'a', 'a']
>>> re.findall('ab+', 'aaaaaaaa')
[]
```

The regex `'ab?'` matches the character `'a'` in the string, followed by character `'b'` if it exists—but it doesn't in the code. The regex `'ab+'` matches the character `'a'` in the string, followed by as many characters `'b'` as possible—but at least one. However, the character `'b'` does not exist so there's no match.

Video

Watch the short chapter video on YouTube:



<https://youtu.be/1Y25qfgjLBo>

Puzzle

```
import re

s = 'xyyy'

print(re.findall('xy?', s)[0])

print(re.findall('xy*', s)[0])

print(re.findall('xy+', s)[0])
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



<https://app.finxter.com/learn/computer/science/568>

Greedy vs Non-Greedy Quantifiers

Ready to earn the black belt of your regex superpower? This chapter shows you the subtle but essential difference between greedy and non-greedy regex quantifiers.

You’ve now mastered three of the most important regex quantifiers. But that’s not all! Let’s dive a bit deeper into the regex quantifiers!

13.1 Regex Quantifiers

The word “quantifier” originates from Latin: its meaning is *quantus* = *how much* / *how often*. This translation accurately describes the meaning of a regular expression quantifier: you tell the regex engine how

often you want to match a given pattern.

If you think you don't define any quantifier, you do it implicitly: no quantifier means to match the regular expression exactly once.

So what are the regex quantifiers in Python (aside from the default *exactly-once* quantifier)?

Quantifier	Meaning
A?	Match pattern A zero or one times
A*	Match pattern A zero or more times
A+	Match pattern A one or more times
A{m}	Match pattern A exactly m times
A{m,n}	Match pattern A between m and n times

You see in the table that the quantifiers `?`, `*`, `+`, `{m}`, and `{m,n}` define how often you repeat the matching of regex A. Let's have a look at some examples—one per quantifier:

```
>>> import re
>>> re.findall('a?', 'aaaa')
['a', 'a', 'a', 'a', '']
>>> re.findall('a*', 'aaaa')
['aaaa', '']
>>> re.findall('a+', 'aaaa')
['aaaa']
```

```
>>> re.findall('a{3}', 'aaaa')
['aaa']
>>> re.findall('a{1,2}', 'aaaa')
['aa', 'aa']
```

In each line, you try a different quantifier on the same text 'aaaa'. And, interestingly, each line leads to a different output:

- The zero-or-one regex 'a?' matches four times one 'a'. Note that it doesn't match zero characters if it can avoid doing so.
- The zero-or-more regex 'a*' matches once four 'a's and consumes them. At the end of the string, it can still match the empty string.
- The one-or-more regex 'a+' matches once four 'a's. In contrast to the previous quantifier, it cannot match an empty string.
- The repeating regex 'a3' matches up to three 'a's in a single run. It can do so only once.
- The repeating regex 'a1,2' matches one or two 'a's. It tries to match as many as possible.

It's time to explore the meaning of the term *greedy*. Shall we?

13.2 Greedy Match

A greedy match means that the regex engine (the one which tries to find your pattern in the string) matches as many characters as possible.

For example, the regex `'a+'` will match as many `'a'`s as possible in your string `'aaaa'`. Although the substrings `'a'`, `'aa'`, `'aaa'` all match the regex `'a+'`, it's not enough for the regex engine. It's always hungry and tries to match even more.

In other words, the greedy quantifiers `?`, `*`, `+`, `{m}`, and `{m,n}` are greedy: they “consume” as many characters as possible while ensuring that the regex pattern is still satisfied if possible. Here are the above examples again that all show how greedy the regex engine is:

```
>>> import re
>>> re.findall('a?', 'aaaa')
['a', 'a', 'a', 'a', '']
>>> re.findall('a*', 'aaaa')
['aaaa', '']
>>> re.findall('a+', 'aaaa')
['aaaa']
>>> re.findall('a{3}', 'aaaa')
['aaa']
```

```
>>> re.findall('a{1,2}', 'aaaa')  
['aa', 'aa']
```

In all cases, a shorter match would also be valid. But as the regex engine is greedy per default, those are not enough for the regex engine.

Okay, so how can you perform a non-greedy match?

13.3 Non-Greedy Match

A non-greedy match means that the regex engine matches as few characters as possible—so that it still can match the pattern in the given string. For example, the regex `'a+?'` will match as few `'a'`s as possible in your string `'aaaa'`. Thus, it matches the first character `'a'` and is done with it. Then, it moves on to the second character (which is also a match) and so on. In other words, the non-greedy quantifiers give you the shortest possible match from a given position in the string. You can make the default quantifiers `?`, `*`, `+`, `{m}`, and `{m,n}` non-greedy by appending a question mark symbol `'?'` to them: `??`, `*?`, `+`, and `{m,n}?`. they “consume” or match as few characters as possible so that the regex pattern is still satisfied. Here are some examples that show how non-greedy matching works:

13.4 Non-Greedy Question Mark (??)

Let's start with the question mark (zero-or-one) quantifier:

```
>>> import re
>>> re.findall('a?', 'aaaa')
['a', 'a', 'a', 'a', '']
>>> re.findall('a??', 'aaaa')
['', 'a', '', 'a', '', 'a', '', 'a', '']
```

In the first instance, you use the zero-or-one regex 'a?'. It's greedy, so it matches one 'a' character if possible. In the second instance, you use the non-greedy zero-or-one version 'a??'. It matches zero 'a's if possible. Note that it moves from left to right, so it matches the empty string, and 'consumes' it. Only then, it cannot match the empty string anymore, so it has to match the first 'a' character. But after that, it's free to match the empty string again. This pattern of first matching the empty string and only then matching the 'a' if it's needed repeats. That's why this strange pattern occurs.

13.5 Non-Greedy Asterisk (*?)

Let's dive into the asterisk (zero-or-more) quantifier:

```
>>> re.findall('a*', 'aaaa')
['aaaa', '']
>>> re.findall('a*?', 'aaaa')
 ['', 'a', '', 'a', '', 'a', '', 'a', '']
```

First, you use the zero-or-more asterisk regex 'a*'. It's greedy, so it matches as many 'a' characters as it can. Second, you use the non-greedy zero-or-one version 'a*?'. Again, it matches zero 'a's, if possible. Only if it has already matched zero characters at a certain position, it matches one character at that position, “consumes” it, and moves on.

13.6 Non-Greedy Plus (+?)

You'll learn about the plus (one-or-more) quantifier next!

```
>>> re.findall('a+', 'aaaa')
['aaaa']
>>> re.findall('a+?', 'aaaa')
['a', 'a', 'a', 'a']
```

First, you use the one-or-more plus regex `'a+'`. Because it's greedy, it matches as many `'a'` characters as it can (but at least one).

Second, you use the non-greedy one-or-more version `'a+?'`. In this case, the regex engine matches only one character `'a'`, consumes it, and moves on with the next match.

Let's summarize what you've learned so far:

13.7 Greedy vs Non-Greedy

Given is a pattern with a quantifier (e.g., the asterisk quantifier) that allows the regex engine to match the pattern multiple times.

A given string may match the regex in multiple ways. For example, both substrings `'a'` and `'aaa'` are valid matches when matching the pattern `'a*'` in the string `'aaaa'`.

So what's the difference between the greedy and the non-greedy match? The greedy one matches as many repetitions of the quantified pattern as possible. The non-greedy one matches as few repetitions of the quantified pattern as possible.

Let's consider a range of examples that help you

understand the difference between greedy and non-greedy matches in Python:

```
>>> import re
>>> re.findall('a+', 'aaaa')
['aaaa']
>>> re.findall('a?', 'aaaa')
['a', 'a', 'a', 'a']
>>> re.findall('a*', 'aaaa')
['aaaa', '']
>>> re.findall('a*?', 'aaaa')
['', 'a', '', 'a', '', 'a', '', 'a', '']
>>> re.findall('a?', 'aaaa')
['a', 'a', 'a', 'a', '']
>>> re.findall('a??', 'aaaa')
 ['', 'a', '', 'a', '', 'a', '', 'a', '']
```

Make sure you entirely understand those examples before you move on. If you don't, please reread the previous paragraphs.

13.8 Which is Faster?

Considering that greedy quantifiers match a maximal and non-greedy a minimal number of patterns, is there any performance difference?

Indeed, some benchmarks suggest that there's a significant performance difference: the greedy quantifier

is 100% slower in realistic experiments on benchmark data. So if you optimize for speed and you don't care about greedy or non-greedy matches—and you don't know anything else—go for the non-greedy quantifier! However, the truth is not as simple. For example, consider the following basic experiment that falsifies the previous hypothesis that the non-greedy version is faster:

```
>>> import timeit
>>> timeit.timeit('import re;re.findall("a*",
↪ "aaaaaaaaaaaa")')
1.0579840000000331
>>> timeit.timeit('import re;re.findall("a*?",
↪ "aaaaaaaaaaaa")')
3.7830938000000742
```

We used the speed testing tool `timeit` that allows you to throw in some simple Python statements and check how long they run. Per default, the statement runs 1,000,000 times. You can see a notable performance difference of more than 300%! The non-greedy version is three times slower than the greedy version. Why is that? The reason is the `re.findall()` method that returns a list of matching substrings. Here's the output both statements would produce:

```
>>> re.findall("a*", "aaaaaaaaaaaa")
['aaaaaaaaaaaa', '']
>>> re.findall("a*?", "aaaaaaaaaaaa")
['', 'a', '', 'a', '', 'a', '', 'a', '', 'a', '',
↪ 'a', '', 'a', '', 'a', '', 'a', '', 'a', '', 'a',
↪ '', 'a', '']
```

You can see that the greedy version finds one match and is done with it. The non-greedy version finds 25 matches! Of course, this leads to far more processing and memory overhead. So what happens if you use the `re.search()` method that returns only the first match rather than the `re.findall()` method that returns all matches?

```
>>> timeit.timeit('import re;re.search("a*",
↪ "aaaaaaaaaaaa")')
0.8420328999998219
>>> timeit.timeit('import re;re.search("a*?",
↪ "aaaaaaaaaaaa")')
0.7955709000000297
```

As expected, this changes things again. Both regex searches yield a single result, but the non-greedy match is much shorter: it matches the empty string `''` rather than the whole string `'aaaaaaaaaaaa'`. Of course, this is a bit faster. However, the difference is negligible in this minimal example.

13.9 Summary

- You've learned that the greedy quantifiers `?`, `*`, and `+` match as many repetitions of the quantified pattern as possible.
- The non-greedy quantifiers `??`, `*?`, and `+`? match as few repetitions of the quantified pattern as possible.
- If you have the flexibility and you care about performance, consider taking the non-greedy version (if this won't end up in a large number of meaningless matches).

Video

Watch the short chapter video on YouTube (Your author here was a bit ill when recording the video ;):



<https://youtu.be/srtn0FXu0vQ>

Puzzle

```
import re

matches = re.findall('finxter+?r', 'finxterrrr')
print(matches[0][:-1])
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



<https://app.finxter.com/learn/computer/science/569>

Line and String Boundaries

This chapter is all about the start of line `^` and end of line `$` regular expressions. These two meta-characters are fundamental to regular expressions—even outside the Python world. So invest 10 minutes now and master them once and for all!

14.1 Start-of-String (`^`)

You can use the caret operator `^` to match the beginning of the string. For example, this is useful if you want to ensure that a pattern appears at the beginning of a string. Here's an example:

```
>>> import re
>>> re.findall('^PYTHON', 'PYTHON is fun.')
```

```
['PYTHON']
```

The caret at the beginning of the pattern `'PYTHON'` ensures that you match the word Python only at the beginning of the string. In the previous example, this doesn't make any difference. But in the next example, it does:

```
>>> re.findall('^PYTHON', 'PYTHON! PYTHON is fun')
['PYTHON']
```

Although there are two occurrences of the substring `'PYTHON'`, there's only one matching substring—at the beginning of the string.

But what if you want to match not only at the beginning of the whole string but also at the beginning of each new line in a multi-line string?

14.2 Start-of-Line (^)

The caret operator, per default, only applies to the start of a string. So if you've got a multi-line string—for example, when reading a text file—it will still only match once: at the beginning of the string. However, you may want to match at the beginning *of each line*.

For example, you may want to find all lines that start with 'Python' in a given string.

You can specify that the caret operator matches the beginning of each line via the `re.MULTILINE` flag. Here's an example showing both usages—without and with setting the `re.MULTILINE` flag:

```
>>> import re
>>> text = '''
Python is great.
Python is the fastest growing
major programming language in
the world.
Pythonistas thrive.'''
>>> re.findall('^Python', text)
[]
>>> re.findall('^Python', text, re.MULTILINE)
['Python', 'Python', 'Python']
>>>
```

The first output is the empty list because the string 'Python' does not appear at the beginning of the string. The second output is the list of three matching substrings because the string 'Python' appears three times at the beginning of a line.

14.3 End of String (\$)

Similarly, you can use the dollar-sign operator `$` to match the end of the string. Here's an example:

```
>>> import re
>>> re.findall('fun$', 'PYTHON is fun')
['fun']
```

The `findall()` method finds all occurrences of the pattern in the string—although the trailing dollar-sign `$` ensures that the regex matches only at the end of the string. This can significantly alter the meaning of your regex as you can see in the next example:

```
>>> re.findall('fun$', 'fun fun fun')
['fun']
```

Although there are three occurrences of the substring `'fun'`, there's only one matching substring—at the end of the string. How can you match at the end of each line in a multi-line string?

14.4 End of Line (\$)

The dollar-sign operator, per default, only applies to the end of a string. So if you've got a multi-line

string—for example, when reading a text file—it will still only match once: at the end of the string. However, you may want to match at the end of each line. For example, you may want to find all lines that end with `'.py'`.

To achieve this, you can specify that the dollar-sign operator matches the end of each line via the `re.MULTILINE` flag. Here's an example showing both usages—without and with setting the `re.MULTILINE` flag:

```
>>> import re
>>> text = '''
Coding is fun
Python is fun
Games are fun
Agreed?'''
>>> re.findall('fun$', text)
[]
>>> re.findall('fun$', text, flags=re.MULTILINE)
['fun', 'fun', 'fun']
>>>
```

The first output is the empty list because the string `'fun'` does not appear at the end of the string. The second output is the list of three matching substrings because the string `'fun'` appears three times at the end of a line.

14.5 Intermezzo: Regex Methods

The `re.sub(pattern, repl, string, count=0, flags=0)` method returns a new string where all occurrences of the pattern in the old string are replaced by `repl`.

You can use the caret operator to substitute wherever some pattern appears at the beginning of the string:

```
>>> import re
>>> re.sub('^Python', 'Code', 'Python is \nPython')
'Code is \nPython'
```

Only the beginning of the string matches the regex pattern so you've got only one substitution. Again, you can use the `re.MULTILINE` flag to match the beginning of each line with the caret operator:

```
>>> re.sub('^Python', 'Code', 'Python is \nPython',
↪ flags=re.MULTILINE)
'Code is \nCode'
```

Now, you replace both appearances of the string `'Python'`.

So you can use the caret operator to match at the beginning of the string. However, you should note that it doesn't make a lot of sense to use it for the `match()` and `fullmatch()` methods as they, by definition, start by trying to match the first character of the string.

You can also use the `re.MULTILINE` flag to match the beginning of each line (rather than only the beginning of the string):

```
>>> text = '''Python is
Python'''
>>> re.findall('^Python', text, flags=re.MULTILINE)
['Python', 'Python']
>>> re.search('^Python', text, flags=re.MULTILINE)
<re.Match object; span=(0, 6), match='Python'>
>>> re.match('^Python', text, flags=re.MULTILINE)
<re.Match object; span=(0, 6), match='Python'>
>>> re.fullmatch('^Python', text, flags=re.MULTILINE)
>>>
```

Again, it's questionable whether this makes sense for the `re.match()` and `re.fullmatch()` methods as they only look for a match at the beginning of the string.

The `re.sub(pattern, repl, string, count=0, flags=0)` method returns a new string where all occurrences of the pattern in the old string are replaced by `repl`. You can use the dollar-sign operator to substitute wherever some pattern appears at the end of the string:

```
>>> import re
>>> re.sub('Python$', 'Code', 'Is Python\nPython')
'Is Python\nCode'
Only the end of the string matches the regex pattern,
↪ so there's only one substitution.
Again, you can use the re.MULTILINE flag to match the
↪ end of each line with the dollar-sign operator:
>>> re.sub('Python$', 'Code', 'Is Python\nPython',
↪ flags=re.MULTILINE)
'Is Code\nCode'
```

Now, you replace both appearances of the string `'Python'`.

All four methods—`re.findall()`, `re.search()`, `re.match()`, and `re.fullmatch()`—search for a pattern in a given string. You can use the dollar-sign operator `$` (or the caret operator `^`) within each pattern to match the end of the string (or the beginning). Here's one example per method:


```
>>> import re
>>> text = 'Python is Python'
>>> re.findall('Python$', text)
['Python']
>>> re.search('Python$', text)
<re.Match object; span=(10, 16), match='Python'>
>>> re.match('Python$', text)
>>> re.fullmatch('Python$', text)
>>>
```

So you can use the dollar-sign operator to match at the end of the string. However, you should note that it doesn't make a lot of sense to use it for the `fullmatch()` methods as it, by definition, already requires that the last character of the string is part of the matching substring.

You can also use the `re.MULTILINE` flag to match the end of each line (rather than only the end of the whole string):

```
>>> text = '''Is Python
Python'''
>>> re.findall('Python$', text, flags=re.MULTILINE)
['Python', 'Python']
>>> re.search('Python$', text, flags=re.MULTILINE)
<re.Match object; span=(3, 9), match='Python'>
>>> re.match('Python$', text, flags=re.MULTILINE)
```

```
>>> re.fullmatch('Python$', text, flags=re.MULTILINE)
>>>
```

As the pattern doesn't match the string prefix, both `re.match()` and `re.fullmatch()` return empty results.

Wow, the longest shaded box we've ever seen!

14.6 Match Caret (^) or Dollar (\$) Symbols

You know that the caret and dollar symbols have a special meaning: they match the beginning or end of each string/line. But what if you search for the caret (^) or dollar (\$) symbols themselves? How can you match them in a string?

Not really to your surprise, the answer is simple: escape the caret or dollar symbols using the backslash symbol. In particular, use `'\^'` instead of `'^'` and `'\$'` instead of `'$'`. Here's an example:

```
>>> import re
>>> text = 'The product ^^^ costs $3 today.'
>>> re.findall('\^', text)
```

```
['^', '^', '^']  
>>> re.findall('\$', text)  
['$']
```

By escaping the special symbols `^` and `$`, you tell the regex engine to ignore their special meaning.

14.7 Summary

You've learned everything you need to know about the caret operator `^` and the dollar-sign operator `$`.

- The caret operator `^` matches at the beginning of a string.
- The dollar-sign operator `$` matches at the end of a string.
- If you want to match at the beginning or end of each line in a multi-line string, you can set the `re.MULTILINE` flag in all the relevant re methods.
- There are more Python regex methods than just `re.findall()`.

Video

Watch the short chapter video on YouTube:



https://youtu.be/-x877_X-wVc

Puzzle

```
import re

text = '''
Tell not me; when the butt is out, we will drink
water; not a drop before: therefore bear up, and
board 'em. Servant-monster, drink to me.
'''

matches = re.findall('^[a-m].*$', text,
    ↪ flags=re.MULTILINE)
print(len(matches))
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



[https://app.finxter.com/learn/
computer/science/571](https://app.finxter.com/learn/computer/science/571)

OR Regex

This chapter is all about the OR | operator.

15.1 What's the OR | Operator?

Given is a string. You want to find all substrings that match either the string 'iPhone' or the string 'iPad'. How can you achieve this?

The easiest way is the Python OR operator | using the regular expression pattern (iPhone|iPad). Here's an example:

```
>>> import re
>>> text = 'Buy now: iPhone only $399 with free iPad'
>>> re.findall('(iPhone|iPad)', text)
['iPhone', 'iPad']
```

The first argument of the `re.findall()` method is the pattern (`iPhone|iPad`). It either matches the first part right in front of the or symbol `|`—which is `iPhone`—or the second part after it—which is `iPad`. The second argument is the sales text `'Buy now: iPhone only $399 with free iPad'`, in which you want to search the pattern. The sales text contains both strings `'iPhone'` and `'iPad'`.

15.2 Examples

Let's study some more examples to teach you all the possible uses and border cases—one after another. You start with the previous example:

```
>>> import re
>>> text = 'Buy now: iPhone only $399 with free iPad'
>>> re.findall('iPhone|iPad', text)
['iPhone', 'iPad']
```

What happens if you don't use the parenthesis?

```
>>> text = 'iPhone iPhone iPhone iPadiPad'
>>> re.findall('iPhone|iPad', text)
['iPhone', 'iPhone', 'iPhone', 'iPad', 'iPad']
>>> re.findall('iPhone|iPad', text)
['iPhone', 'iPhone', 'iPhone', 'iPad', 'iPad']
```

In the second example, you just skipped the parentheses using the regex pattern `iPhone|iPad` rather than `(iPhone|iPad)`. But no problem—it still works and generates the exact same output! But what happens if you leave one side of the OR operation empty?

```
>>> re.findall('iPhone|', text)
['iPhone', '', 'iPhone', '', 'iPhone', '', '', '',
↪  '', '', '', '', '', '', '']
```

The output is not as strange as it seems. The OR operator allows for empty operands—in which case it wants to match the non-empty string. If this is not possible, it matches the empty string (so everything will be a match). The previous example also shows that it still tries to match the non-empty string if possible. But what if the trivial empty match is on the left side of the OR operand?

```
>>> re.findall('|iPhone', text)
['', 'iPhone', '', '', 'iPhone', '', '', 'iPhone',
↪  '', '', '', '', '', '', '', '']
```

The example shows some subtleties of the regex engine. First of all, it still matches the non-empty string, if possible! But more importantly, you can see that the regex engine matches from left to right.

It first tries to match the left regex (which it is always possible for every single position in the text). An empty string that's already matched will not be considered anymore. Only then, it tries to match the regex on the right side of the OR operator.

Think of it this way: the regex engine moves from the left to the right—one position at a time. It matches the empty string every single time. Then it moves over the empty string, and in some cases, it can still match the non-empty string. Each match “consumes” a substring and cannot be matched any more. That's why you see the first match is the empty string, and the second match is the substring `'iPhone'`.

15.3 Nested OR

Okay, you're not easily satisfied, are you? Let's try nesting the Python regex OR operator `|`.

```
>>> text = 'xxx iii zzz iii ii xxx'
>>> re.findall('xxx|iii|zzz', text)
['xxx', 'iii', 'zzz', 'iii', 'xxx']
```

So you can use multiple OR operators in a row. Of course, you can also use the grouping (parentheses) operator to nest an arbitrary complicated construct of OR operations:

```
>>> re.findall('x(i|(zz|ii|(x| )))', text)
[('x', 'x', 'x'), (' ', ' ', ' '), ('x', 'x', 'x')]
```

But this seldom leads to clean and readable code. You can avoid this easily by putting a bit of thought into your regex design.

15.4 Character Set OR

If you only want to match a single character out of a set of characters, the character set is a much better way of doing it:

```
>>> import re
>>> text = 'hello world'
>>> re.findall('[abcdefghijklmnopqrstuvwxyz]+', text)
['hello', 'world']
```

A shorter and more concise version would be to use the range operator within character sets:

```
>>> re.findall('[a-z]+', text)
['hello', 'world']
```

You enclose the character set in the square brackets [] to match exactly one of the symbols. Thus, it carries the same semantics as the OR operator: |. However, if you try to do something on those lines...

```
>>> re.findall('(a|b|c|d|e|f|g|h|i|j|k|l|m
↪ |n|o|p|q|r|s|t|u|v|w|x|y|z)+', text)
['o', 'd']
```

... you'll write much less concise code and risk of getting confused by the output. The reason is that the parenthesis is the group operator—it captures the position and substring that matches the regex. Used in the `findall()` method, it only returns the content of the last matched group. This turns out to be the last character of the word 'hello' and the final character of the word 'world'.

15.5 Match the Vertical Line '|'

So if the character '|' stands for the or character in a given regex, the question arises on how to match the vertical line symbol '|' itself? By now, you probably know what to do (but we'll still tell you - repetition is king): use the backslash. In particular, use 'A\\B' instead of 'A|B' to match the string 'A|B' itself. Here's an example:

```
>>> import re
>>> re.findall('A|B', 'AAAA|BBBB')
['A', 'A', 'A', 'A', 'B', 'B', 'B', 'B']
>>> re.findall('A\\B', 'AAAA|BBBB')
['A|B']
```

Do you really get the outputs of this code snippet? In the first example, you're searching for either character 'A' or character 'B'. In the second example, you're searching for the string 'A|B' (which contains the '|' character).

15.6 Summary

- The OR pattern 'X|Y' matches either pattern X or pattern Y.
- You can nest the OR pattern 'X|(Y|Z)', but it will hurt the readability of your code.
- You should use the character class if your OR operands are simple characters.
- Don't forget to escape the OR symbol if you want to match the vertical line.

Video

Watch the short chapter video on YouTube:



<https://youtu.be/wx9PGvSxQRs>

Puzzle

```
import re

text = '''
Tell not me; when the butt is out, we will drink
water; not a drop before: therefore bear up, and
board 'em. Servant-monster, drink to me.
'''

matches = re.findall('not|me', text)
print(len(matches))
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



<https://app.finxter.com/learn/computer/science/572>

AND Regex

If there's an OR regex, there must also be an AND operator, right?

This chapter is all about the AND operator of Python's re library. You may ask: what? (And rightly so.) Sure, there's the OR operator (example: `'iPhone|iPad'`). But what's the meaning of matching one regular expression AND another?

There are different interpretations for the AND operator in a regular expression (regex):

- Ordered: Match one regex pattern after another. In other words, you first match pattern A, *and* then you match pattern B. Here, the answer is simple: you use the pattern AB to match

both.

- Unordered: Match multiple patterns in a string but in no particular order. In this case, you'll use a bag-of-words approach.

We'll discuss both in the following.

16.1 Ordered Regex AND Operator

Given is a string. Say, your goal is to find all substrings that match string 'iPhone', followed by string 'iPad'. You can view this as the AND operator of two regular expressions. How can you achieve this?

The straightforward AND operation of both strings is the regular expression pattern `iPhoneiPad`. In the following example, you want to match pattern 'aaa' and pattern 'bbb'—in this order.

```
>>> import re
>>> text = 'aaabaaaabbb'
>>> A = 'aaa'
>>> B = 'bbb'
>>> re.findall(A+B, text)
['aaabbb']
>>>
```

You use the `re.findall()` method. The first argument is the pattern `A+B` which evaluates to `'aaabbb'`. There's nothing fancy about this: each time you write a string consisting of more than one character, you essentially use the ordered AND operator.

The second argument is the text `'aaabaaaabbb'`, which you want to search for the pattern. The result shows that there's a matching substring in the text: `'aaabbb'`.

16.2 Unordered Regex AND Operator

But what if you want to search a given text for pattern `A` AND pattern `B`—but in no particular order? In other words: if both patterns appear anywhere in the string, the whole string should be a match.

Now, this is a bit more complicated because any regular expression pattern reads from left to right. A simple solution is to use the lookahead assertion `'(?.*A)'` to check whether regex `A` appears anywhere in the string. (You assume a single line string as the `.*` pattern doesn't match the newline character by default.) Let's first have a look at the minimal solution to check for two patterns anywhere in the string (say,

16.2. UNORDERED REGEX AND OPERATOR 43

patterns 'hi' AND 'you').

```
>>> import re
>>> pattern = '(?=. *hi)(?=. *you)'
>>> re.findall(pattern, 'hi how are yo?')
[]
>>> re.findall(pattern, 'hi how are you?')
['']
```

In the first example, both words do not appear. In the second example, they do.

But how does the lookahead assertion work? You must know that any other regex pattern “consumes” the matched substring. No other part of the regex can match the once consumed substring. Think of the lookahead assertion as a non-consuming pattern match. The regex engine goes from the left to the right—searching for the pattern. At each point, it has one “current” position to check if this position is the first position of the remaining match. In other words, the regex engine tries to “consume” the next character as a (partial) match of the pattern. The advantage of the lookahead expression is that it doesn’t consume anything. It just “looks ahead” starting from the current position, whether what follows would theoretically match the lookahead pattern. If it doesn’t, the regex engine cannot move on. A simple example

of lookahead. The regular expression engine matches (“consumes”) the string partially. Then it checks whether the remaining pattern could match without actually matching it.

Let’s go back to the expression `(?=.*hi)(?=.*you)` to match strings that contain both `'hi'` and `'you'`. Why does it work? The reason is that the lookahead expressions don’t consume anything. You first search for an arbitrary number of characters `.*`, followed by the word `hi`. But because the regex engine hasn’t consumed anything, it’s still at the same position at the beginning of the string. So, you can repeat the same for the word `you`. Note that this method doesn’t care about the order of the two words:

```
>>> import re
>>> pattern = '(?=.*hi)(?=.*you)'
>>> re.findall(pattern, 'hi how are you?')
['']
>>> re.findall(pattern, 'you are how? hi!')
['']
```

No matter which word `'hi'` or `'you'` appears first in the text, the regex engine finds both. You may ask: why is the output the empty string? The reason is that the regex engine hasn’t consumed any character.

It just checked the lookaheads. So the easy fix is to consume all characters as follows:

```
>>> import re
>>> pattern = '(?=. *hi)(?=. *you). *'
>>> re.findall(pattern, 'you fly high')
['you fly high']
```

Now, the whole string is a match because after checking the lookahead with `'(?=. *hi)(?=. *you)'`, you also consume the whole string `'. *'`.

16.3 Summary

There are different interpretations for the AND operator in a regular expression (regex):

- **Ordered:** Match one regex pattern after another. In other words, you first match pattern A, and then you match pattern B. Here the answer is simple: you use the pattern `AB` to match both.
- **Unordered:** Match multiple patterns in a string but in no particular order. In this case, you'll use a bag-of-words approach.

Video

Watch the short chapter video on YouTube:



<https://youtu.be/r9Gaauf1Qk>

Puzzle

```
import re

text = '''
Tell not me; when the butt is out, we will drink
water; not a drop before: therefore bear up, and
board 'em. Servant-monster, drink to me.
'''

pattern = '(?=. *drink)(?=. *you)(?=. *me)'
matches = re.findall(pattern, text)
all_terms_in_line = len(matches) > 0
print(all_terms_in_line)
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



[https://app.finxter.com/learn/
computer/science/573](https://app.finxter.com/learn/computer/science/573)

NOT Regex

How can you search a string for substrings that do NOT match a given pattern? In other words, what’s the “negative pattern” in Python’s regular expressions?

The answer is two-fold: If you want to match all characters except a set of specific characters, you can use the negative character set `[^...]`. If you want to match all substrings except the ones that match a regex pattern, you can use the feature of negative lookahead `(?!...)`. Here’s an example for the negative character set:

```
>>> import re
>>> re.findall('[^a-m]', 'aaabbbbaababmmmmnooppmaa')
['n', 'o', 'o', 'p']
```

And here’s an example for the negative lookahead pattern to match all “words that are not followed by words”:

```
>>> re.findall('[a-z]+(?![a-z]+)', 'hello world')
['hello', 'world']
```

The negative lookahead `(?![a-z]+)` doesn’t consume (match) any character. It just checks whether the pattern `[a-z]+` does NOT match at a given position. The only time this happens is just before the space character and the end of the string.

Video

Watch the short chapter video on YouTube:



<https://youtu.be/RFyzd9xP7cM>

Puzzle

```
import re
```

```
pattern = '[a-z]+(?! world)'  
string = 'hello world'  
print(re.findall(pattern, string))
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



[https://app.finxter.com/learn/
computer/science/574](https://app.finxter.com/learn/computer/science/574)

Matching Groups

This chapter explains everything you need to know about matching groups of regular expressions. You may have also read the term *capture groups*, which points to the same concept.

So let's start with the basics:

18.1 Simple Matching Groups

As you use parentheses to structure mathematical expressions, $(2 + 2) * 2$ versus $2 + (2 * 2)$, you use parentheses to structure regular expressions. An example regex that does this is `'a(b|c)'`. The whole content enclosed in the opening and closing parentheses is called *matching group* (or *capture group*). You

can have multiple matching groups in a single regex. And you can even have hierarchical matching groups, for example `'a(b|(cd))'`.

A significant advantage of a matching group is that it captures the matched substring. In other words, you can retrieve it in later parts of the regular expression pattern—or even after analyzing the overall result to answer questions such as *what parts of the string matched the matching group?*

Let's have a short example for the most basic use of a matching group—to structure the regex.

Say you create the regex `b?(a.)*` with the matching group `(a.)` that matches all patterns starting with zero or one occurrence of character `'b'` and an arbitrary number of two-character-sequences starting with the character `'a'`. Hence, the strings `'bacacaca'`, `'aaaa'`, `''` (the empty string), and `'Xababababab'` all match your regex.

The use of the parentheses for structuring the regular expression is intuitive and should come naturally to you because the same rules apply as for arithmetic operations. However, there's a more advanced use of regex groups: *retrieval*. You can retrieve the matched content of each matching group. So the next question naturally arises:

18.2 First Matching Group

There are two scenarios when you want to access the content of your matching groups:

- Access the matching group in the regex pattern to reuse partially matched text from one group somewhere else.
- Access the matching group after the whole match operation to analyze the matched text in your Python code.

In the first case, you simply get the first matching group with the `\number` special sequence. For example, to get the first matching group, you'd use the `\1` special sequence. Here's an example:

```
>>> import re
>>> re.search(r'(j.n) is \1', 'jon is jon')
<re.Match object; span=(0, 10), match='jon is jon'>
```

You'll use this feature a lot because it gives you much more expression power: for example, you can search for a name in a text-based on a given pattern and then process specifically this name in the rest

of the text (and not all other names that would also match the pattern).

Note that the numbering of the groups begins with `\1` and not with `\0`.

In the second case, you want to know the contents of the first group after the whole match. How do you do that?

The answer is also simple: use the `m.group(0)` method on the matching object `m`. Here's an example:

```
>>> import re
>>> m = re.search(r'(j.n)', 'jon is jon')
>>> m.group(1)
'jon'
```

The numbering works consistently with the previously introduced regex group numbering: start with identifier 1 to access the contents of the first group.

18.3 Other Matching Groups

Again, there are two different intentions when asking this question:

- Access the matching group in the regex pattern

to reuse partially matched text from one group somewhere else.

- Access the matching group after the whole match operation to analyze the matched text in your Python code.

In the first case, you use the special sequence `\2` to access the second matching group, `\3` to access the third matching group, and `\99` to access the ninety-ninth matching group. Here's an example:

```
>>> import re
>>> re.search(r'(j..) (j..)\s+\2', 'jon jim jim')
<re.Match object; span=(0, 11), match='jon jim jim'>
>>> re.search(r'(j..) (j..)\s+\2', 'jon jim jon')
>>>
```

As you can see, the sequence `\2` refers to the matching contents of the second group `'jim'`.

In the second case, you can simply increase the identifier too to access the other matching groups in your Python code:

```
>>> import re
>>> m = re.search(r'(j..) (j..)\s+\2', 'jon jim
↪ jim')
>>> m.group(0)
```

```
'jon jim jim'  
>>> m.group(1)  
'jon'  
>>> m.group(2)  
'jim'
```

This code also shows an interesting feature: if you use the identifier 0 as an argument to the `m.group(0)` method, the `regex` module will give you the contents of the whole match. You can think of it as the first group being the entire match.

18.4 Named Groups

Accessing the captured group using the notation `\number` is not always convenient and sometimes not even possible (for example, if you have more than 99 groups in your regex). A significant disadvantage of regular expressions is that they tend to be hard to read. It's, therefore, important to know about the different tweaks to improve readability.

One such optimization is a named group, which is a matching group that captures part of the match but with one twist: it has a name. Now, you can use this name to access the captured group at a later point in your regular expression pattern. This can improve the readability of your regular expressions.

```
import re
pattern = '(?P<quote>["\']).*?(?P=quote)'
text = 'She said "hi"'
print(re.search(pattern, text))
# <re.Match object; span=(9, 13), match='"hi"'>
```

The code searches for substrings enclosed in either single or double quotes. You match the opening quote by using the regex `[\"']`. You escape the single quote, `textquotesingle`, so that the Python regex engine does not assume (wrongly) that the single quote indicates the end of the string. Then, you use the same group to match the closing quote of the same character which can be either a single or double quote.

18.5 Non-Capturing Groups

In the previous examples, you've seen how to match and capture groups with the parentheses `(...)`. You've learned that each match of this basic group operator is captured so that you can retrieve it later in the regex with the special commands `\2`, `\3`, ..., `\99` or after the match on the matched object `m` with the method `m.group(1)`, `m.group(2)`, and so on.

But what if you don't need it? What if you just need to keep your regex pattern in order—but you don't want to capture the content of a matching group?

The simple solution is the non-capturing group operation `(?: ... )`. You can use it just like the capturing group operation `( ... )`. Here's an example:

```
>>>import re
>>> re.search('(?:python|java) is great', 'python is
↳ great. java is great.')
<re.Match object; span=(0, 15), match='python is
↳ great'>
```

The non-capturing group exists with the sole purpose to structure the regex. You cannot use its content later:

```
>>> m = re.search('(?:python|java) is great', 'python
↳ is great. java is great.')
>>> m.group(1)
Traceback (most recent call last):
  File "<pyshell#28>", line 1, in <module>
    m.group(1)
IndexError: no such group
>>>
```

If you try to access the content of the non-capturing group, the regex engine will throw an `IndexError: no such group`.

Of course, there's a straightforward alternative to non-capturing groups. You can simply use the regular (capturing) group but don't access its content. Only rarely will the performance penalty of capturing a group that's not needed, have any meaningful impact on your overall application.

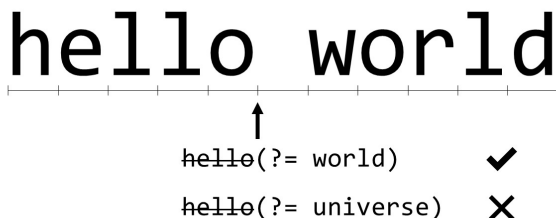
18.6 Positive Lookahead

The concept of lookahead is a compelling one, and any advanced coder should know it. Recently, a friend mentioned that he had written a complicated regex that ignores the order of occurrences of two words in a given text. It's a challenging problem, and without the concept of lookahead, the resulting code will be complicated and hard to understand. However, the lookahead makes this problem simple to write and read.

But first things first: how does the lookahead assertion work? The regex engine moves from left to right, consuming all partially matching substrings. No other part of the regex can match these consumed substrings again.

The regular expression engine matches ("consumes") the string partially (see Figure 18.1). Then it checks

Figure 18.1: A simple example of lookahead.



whether the remaining pattern could be matched without actually matching it.

Think of the lookahead assertion as a non-consuming pattern match. The regex engine moves from left to right—searching for the pattern. At each point, it has one current position to check if this position is the first position of the remaining match. In other words, the regex engine tries to consume the next character as a (partial) match of the pattern.

The advantage of the lookahead expression is that it doesn't consume anything. It just *looks ahead* starting from the current position, checking whether the following characters would theoretically match the looka-

head pattern. If it doesn't, the regex engine cannot move on. Next, it “backtracks”—which is just a fancy way of saying: it goes back to a previous decision and tries to match something else.

18.7 Negative Lookahead

The negative lookahead works just like the positive lookahead (see Chapter 16.2)—only it checks that the given regex pattern does not occur going forward from a certain position. Here's an example:

```
>>> import re
>>> re.search('(?!.*hi.*)', 'hi say hi?')
<re.Match object; span=(8, 8), match=''>
```

The negative lookahead pattern `(?!.*hi.*)` ensures that, by going forward in the string, there's no occurrence of the substring `'hi'`. The first position where this holds true is position 8 (right after the second `'h'`). Like the positive lookahead, the negative lookahead does not consume any character, so the result is the empty string (which is a valid match of the pattern). You can even combine multiple negative lookaheads like this:

```
>>> re.search('(?!.*hi.)(?!\\?).*', 'hi say hi?')
<re.Match object; span=(8, 9), match='i'>
```

You search for a position in the string where 'hi' is not in the lookahead, and the question mark character does not follow immediately afterward. This time, you consume an arbitrary character, so the resulting match is the character 'i'.

18.8 Group Flags

You can control the regex engine with the flags argument of the `re.findall()`, `re.search()`, or `re.match()` methods. For example, to ignore capitalization of your matched substring, pass the `re.IGNORECASE` flag to the regex methods:

```
>>> re.findall('PYTHON', 'python is great',
↳ flags=re.IGNORECASE)
['python']
```

But using a global flag for the whole regex is not always optimal. What if you want to ignore the capitalization only for a certain subregex? You can do this with the group flags: `a`, `i`, `L`, `m`, `s`, `u`, and `x`. Each group flag has its own meaning:

- a If you don't use this flag, the special Python regex symbols `\w`, `\W`, `\b`, `\B`, `\d`, `\D`, `\s` and `\S` will match Unicode characters. If you use this flag, those special symbols will match only ASCII characters — as the name suggests.
- i If you use this flag, the regex engine will perform case-insensitive matching. So if you're searching for `[A-Z]`, it will also match `[a-z]`.
- L Don't use this flag – ever. It's deprecated—the idea was to perform case-insensitive matching depending on your current locale. But it isn't reliable.
- m This flag switches on the following feature: the start-of-the-string regex `^` matches at the beginning of each line (rather than only at the beginning of the string). The same holds for the end-of-the-string regex `$` that now matches also at the end of each line in a multi-line string.
- s Without using this flag, the dot regex `.` matches all characters except the newline character `\n`. Switch on this flag to really match all characters including the newline character.
- x To improve the readability of complicated regular expressions, you may want to allow com-

ments and (multi-line) formatting of the regex itself. This is possible with this flag: all whitespace characters and lines that start with the character '#' are ignored in the regex.

For example, if you want to switch off the differentiation of capitalization, use the `i` flag as follows:

```
>>> re.findall('(?i:PYTHON)', 'python is great')
['python']
```

You can also switch off the capitalization for the whole regex with the “global group flag” `(?i)` as follows:

```
>>> re.findall('(?i)PYTHON', 'python is great')
['python']
```

18.9 Summary

- You’ve learned about matching groups to structure the regex and capture parts of the matching result.
- You can then retrieve the captured groups with the `\number` syntax within the regex pattern

itself and with the `m.group(i)` syntax in the Python code at a later stage.

- Lookahead assertions are non-consuming pattern matches. The regex engine goes from the left to the right—searching for the pattern. At each point, it has one current position to check if this position is the first position of the remaining match.

Video

Watch the short chapter video on YouTube:



<https://youtu.be/JwkciuqcDH4>

Puzzle

```
import re

decode = {'112': 'S', '121': 'B', '211': 'P',
          '122': 'C', '221': 'E', '212': 'F',
          '222': 'G', '111': 'H', '1111': 'I',
          '1112': 'Y', '12': 'X', '1211': 'L',
```

```
        '2111': 'T', '1122': 'N', '1221':  
        ↪ '0\textquotesingle{}}  
code = '0211011120211101110122101122\  
0110111101120110122012210122101211'  
regex = '0([1|2]+)'  
  
for m in re.finditer(regex, code):  
    print(decode.get(m.group(1), ' '), end='')
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



[https://app.finxter.com/learn/
computer/science/557](https://app.finxter.com/learn/computer/science/557)

Split Method

This chapter is all about the `re.split(pattern, string)` method of Python's `re` library.

19.1 `re.split()`

The `re.split(pattern, string, maxsplit=0, flags=0)` method returns a list of strings by matching all occurrences of the pattern in the string and dividing the string along those.

Here's a minimal example:

```
>>> import re
>>> string = 'Learn Python with\t    Finxter!'
>>> re.split('\s+', string)
['Learn', 'Python', 'with', 'Finxter!']
```

The string contains four words that are separated by whitespace characters (in particular: the empty space ' ' and the tabular character '\t'). You use the regular expression '\s+' to match all occurrences of a positive number of subsequent whitespaces. The matched substrings serve as delimiters. The result is the string divided along those delimiters. But that's not all! Let's have a look at the formal definition of the split method.

Specification:

```
re.split(pattern, string, maxsplit=0, flags=0)
```

The method has four arguments—two of which are optional.

- **pattern**: the regular expression pattern you want to use as a delimiter.
- **string**: the text you want to break up into a list of strings.
- **maxsplit** (optional argument): the maximum number of split operations (= the size of the returned list). Per default, the **maxsplit** argument is 0, which means that it's ignored.

- **flags** (optional argument): a more advanced modifier that allows you to customize the behavior of the function. Per default, the regex module does not consider any flags.

The first and second arguments are required. The third and fourth arguments are optional. You'll learn about those arguments in more detail later.

Return Value: The regex split method returns a list of substrings obtained by using the regex as a delimiter.

19.2 Minimal Example

Let's study some more examples—from simple to more complex. The most straightforward use is with only two arguments: the delimiter regex and the string to be split.

```
>>> import re
>>> string = 'fgfffffgfgPythonfgisfffawesomefgffg'
>>> re.split('[fg]+', string)
['', 'Python', 'is', 'awesome', '']
```

You use an arbitrary number of 'f' or 'g' characters as regular expression delimiters. How do you accomplish this? By combining the character class regex

[A] and the one-or-more regex `A+` into the following regex: `[fg]+`. The strings in between are added to the return list.

19.3 Maxsplit Argument

What if you don't want to split the whole string but only a limited number of times. Here's an example:

```
>>> string =
↪ 'a-bird-in-the-hand-is-worth-two-in-the-bush'
>>> re.split('-', string, maxsplit=5)
['a', 'bird', 'in', 'the', 'hand',
↪ 'is-worth-two-in-the-bush']
>>> re.split('-', string, maxsplit=2)
['a', 'bird', 'in-the-hand-is-worth-two-in-the-bush']
```

You use the simple delimiter regex `'-'` to divide the string into substrings. In the first method call, you set `maxsplit=5` to obtain six list elements. In the second method call, you set `maxsplit=3` to obtain three list elements. Can you see the pattern?

You can also use positional arguments to save some characters:

```
>>> re.split('-', string, 2)
['a', 'bird', 'in-the-hand-is-worth-two-in-the-bush']
```

But as many coders don't know about the `maxsplit` argument, you probably should use the keyword argument for readability.

19.4 Optional Flag Argument

As you've seen in the specification, the `re.split()` method comes with an optional fourth 'flag' argument:

```
re.split(pattern, string, maxsplit=0, flags=0)
```

Here's how you'd use it in practice:

```
>>> import re
>>> re.split('[xy]+', text, flags=re.I)
['the', 'russians', 'are', 'coming']
```

Although your regex is lowercase, you ignore the capitalization by using the flag `re.I` which is short for `re.IGNORECASE`. If you wouldn't do it, the result would be quite different:

```
>>> re.split('[xy]+', text)
['theXXXXYYrussiansXX', 'are', 'Y', 'coming']
```

As the character class `[xy]` only contains lowercase characters 'x' and 'y', their uppercase variants appear in the returned list rather than being used as delimiters.

19.5 Regex Versus String Split

The method `re.split()` is much more powerful. The `re.split(pattern, string)` method can split a string along with all occurrences of a matched pattern. The pattern can be arbitrarily complicated. This is in contrast to the `string.split(delimiter)` method, which also splits a string into substrings along the delimiter. However, the delimiter must be a normal string.

An example where the more powerful `re.split()` method is superior is in splitting a text along any whitespace characters:

```
import re

text = '''
    Ha! let me see her: out, alas! he's cold:
    Her blood is settled, and her joints are stiff;
    Life and these lips have long been separated:
    Death lies on her like an untimely Frost
    Upon the sweetest flower of all the field.
```

```

'''
print(re.split('\s+', text))
'''
['', 'Ha!', 'let', 'me', 'see', 'her:', 'out,',
↪ 'alas!',
'he's", 'cold:', 'Her', 'blood', 'is', 'settled,',
↪ 'and',
'her', 'joints', 'are', 'stiff;', 'Life', 'and',
↪ 'these',
'lips', 'have', 'long', 'been', 'separated:',
↪ 'Death',
'lies', 'on', 'her', 'like', 'an', 'untimely',
↪ 'Frost',
'Upon', 'the', 'sweetest', 'flower', 'of', 'all',
↪ 'the',
'field.', '']
'''

```

The `re.split()` method divides the string along any positive number of whitespaces. You couldn't achieve such a result with `string.split(delimiter)` because the delimiter must be a constant-sized string.

19.6 Summary

You've learned about the `re.split(pattern, string)` method that divides the string along the matched pattern occurrences and returns a list of substrings.

Video

Watch the short chapter video on YouTube:



https://youtu.be/sWjU442_eYQ

Puzzle

```
import re

regex = "e."
string = "Spiderman"
result = re.split(regex, string)

print(result[1])
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



<https://app.finxter.com/learn/computer/science/551>

Substitution Method

Do you want to replace all occurrences of a pattern in a string? This chapter is all about the `re.sub(pattern, string)` method of Python's `re` library. Let's answer the following question:

20.1 `re.sub()`

The `re.sub(pattern, repl, string, count=0, flags=0)` method returns a new string where all occurrences of the pattern in the old string are replaced by `repl`. Here's a minimal example:

```
>>> import re
>>> text = 'C++ is the best language. C++ rocks!'
>>> re.sub('C\\+\\+', 'Python', text)
```

```
'Python is the best language. Python rocks!'  
>>>
```

The text contains two occurrences of the string 'C++'. You use the `re.sub()` method to search all of those occurrences. Your goal is to replace all those with the new string 'Python' (Python is the best language after all :)). Note that you must escape the '+' symbol in 'C++' as otherwise it would mean the *at-least-one* regex.

You can also see that the `sub()` method replaces all matched patterns in the string—not only the first one. But there's more! Let's have a look at the formal definition of the `sub()` method.

Specification:

```
re.sub(pattern, repl, string, count=0, flags=0)
```

The method has five arguments—two of which are optional.

- **pattern**: the regular expression pattern to search for strings you want to replace.
- **repl**: the replacement string or function. If it's a function, it must be one that takes one

argument (the match object), which is passed for each occurrence of the pattern.

- **string**: the text you want to replace.
- **count** (optional argument): the maximum number of replacements you want to perform. Per default, you use `count=0`, which reads as replace all occurrences of the pattern.
- **flags** (optional argument): a more advanced modifier that allows you to customize the behavior of the method. Per default, you don't use any flags.

The initial three arguments are required. The remaining two arguments are optional.

Return Value: A new string where count occurrences of the first substrings that match the pattern are replaced with the string value defined in the `repl` argument.

20.2 Minimal Example

Let's study some more examples—from simple to more complex. The most straightforward use is with only

three arguments: the pattern 'sing', the replacement string 'program', and the string you want to modify (text in our example).

```
>>> import re
>>> text = 'Learn to sing because singing is fun.'
>>> re.sub('sing', 'program', text)
'Learn to program because programing is fun.'
```

Just ignore the grammar mistake for now. You get the point: you don't sing, you code.

But what if you actually want to fix this grammar mistake? After all, it's 'programming', not 'programing'. In this case, you need to substitute 'sing' with 'program' in some cases and 'sing' with 'programm' in other cases.

You see where this leads us: the sub-argument must be a function! So let's try this:

```
import re

def sub(matched):
    if matched.group(0)=='singing':
        return 'programming'
    else:
        return 'programm'
```

```
text = 'Learn to sing because singing is fun.'
print(re.sub('sing(ing)?', sub, text))
# Learn to program because programming is fun.
```

In this example, you first define a substitution function `sub`. The function takes the matched object as an input and returns a string. If it matches the longer form `'singing'`, it returns `'programming'`. Else, it matches the shorter form `'sing'`, so it returns the shorter replacement string `'program'` instead.

20.3 count Argument

What if you don't want to substitute all occurrences of a pattern but only a limited number of them? Just use the `count` argument! Here's an example:

```
>>> import re
>>> s = 'xxxxxxhelloworld!xxxx'
>>> re.sub('x+', '', s, count=2)
'helloworld!xxxx'
>>> re.sub('x+', '', s, count=3)
'helloworld!'
```

In the first substitution operation, you replace only two occurrences of the pattern `'x+'`. In the second,

you replace all three. You can also use positional arguments to save some characters:

```
>>> re.sub('x+', '', s, 3)
'helloworld!'
```

But because many coders don't know about the count argument, you probably should use the key-word argument for readability.

20.4 Optional Flag Argument

As you've seen in the specification, the `re.sub()` method comes with an optional fourth flag argument:

```
re.sub(pattern, repl, string, count=0, flags=0)
```

Flags allow you to control the regular expression engine. Because regular expressions are so powerful, they are a useful way of switching on and off certain features (for example, whether to ignore capitalization when matching your regex). Here's how you'd use the flags argument in a minimal example:

```
>>> import re
>>> s = 'xxxiixxxXxxxiixXXX'
>>> re.sub('x+', '', s)
```

```
'iiiXiiiXXX'
>>> re.sub('x+', '', s, flags=re.I)
'iiiiii'
```

In the second substitution operation, you ignore the capitalization by using the flag `re.I`. It substitutes even the uppercase 'X' characters that now match the regex 'x+', too.

20.5 Regex Sub Versus String Replace

In a way, the `re.sub()` method is the more powerful variant of the `string.replace()` method. Why? Because you can replace all occurrences of a regex pattern rather than only all occurrences of a string in another string. So with `re.sub()` you can do everything you can do with `string.replace()` but some things more! Here's an example:

```
>>> 'Python is python is PYTHON'.replace('python',
↪ 'fun')
'Python is fun is PYTHON'
>>> re.sub('(Python)|(python)|(PYTHON)', 'fun',
↪ 'Python is python is PYTHON')
'fun is fun is fun'
```

The `string.replace()` method only replaces the lowercase word 'python' while the `re.sub()` method replaces all occurrences of uppercase or lowercase variants.

Note, you can accomplish the same thing even more comfortably with the `flags` argument.

```
>>> re.sub('python', 'fun', 'Python is python is
↪ PYTHON', flags=re.I)
'fun is fun is fun'
```

20.6 Remove Regex Pattern

Nothing simpler than that. Just use the empty string as a replacement string:

```
>>> re.sub('p', '', 'Python is python is PYTHON',
↪ flags=re.I)
'ython is ython is YTHON'
```

You replace all occurrences of the pattern 'p' with the empty string ''. In other words, you remove all occurrences of 'p'. As you use the `flags=re.I` argument, you ignore capitalization.

20.7 Summary

You've learned the `re.sub(pattern, repl, string, count=0, flags=0)` method returns a new string where all occurrences of the pattern in the old string are replaced by `repl`.

Video

Watch the short chapter video on YouTube:



<https://youtu.be/CAwIa8ngYI8>

Puzzle

```
import re

text = '''I'm not happy.
I don't want to dance.
I don't love ice cream.'''

pattern = "not |don't "
print(re.sub(pattern, '', text))
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



[https://app.finxter.com/learn/
computer/science/578](https://app.finxter.com/learn/computer/science/578)

Compile Method

In this chapter, we'd like to show you a more advanced `re` method: `re.compile(pattern)`.

21.1 `re.compile()`

The `re.compile(pattern)` method returns a match object.

You use this object to call important regex methods you already know such as `search(string)`, `match(string)`, `fullmatch(string)`, and `findall(string)`. In short: You compile the pattern first, then you search the pattern in a string.

This two-step approach is more efficient than calling, say, `search(pattern, string)` at once. That is

if you call the `search()` method multiple times on the same pattern. Why? Because you can reuse the compiled pattern multiple times.

Here's an example:

```
import re

# These two lines ...
regex = re.compile('Py...n')
match = regex.search('Python is great')

# ... are equivalent to ...
match = re.search('Py...n', 'Python is great')
```

In both instances, the `match` variable contains the following match object:

```
<re.Match object; span=(0, 6), match='Python'>
```

But in the first case, you can find the pattern not only in the string `'Python is great'` but also in other strings—without any redundant work of compiling the pattern again and again.

Specification:

```
re.compile(pattern, flags=0)
```

The method has up to two arguments.

- `pattern`: the regular expression pattern that you want to match.
- `flags` (optional argument): a more advanced modifier that allows you to customize the behavior of the function.

We'll explore those arguments in more detail later.

Return Value: The `re.compile(patterns, flags)` method returns a regular expression object. You may ask (and rightly so):

21.2 Regular Expression Object

A regex object is a compiled representation of the pattern. It allows you to access all the different methods such as `findall()` and `search()`. In fact, the regex method `re.search(pattern, string)` internally compiles a regex object `p` and then call `p.search(string)`.

You can see this fact in the official implementation of the `re.search(pattern, string)` method (source: GitHub repository of the `re` package):

```
def search(pattern, string, flags=0):  
    """Scan through string looking for a match to the  
    ↪ pattern, returning  
    a Match object, or None if no match was found."""  
    return _compile(pattern, flags).search(string)
```

The `re.search(pattern, string)` method is a mere wrapper for compiling the pattern first and calling the `p.search(string)` function on the compiled regex object `p`.

21.3 Discussion

Is It Worth Using Python's `re.compile()`? No, in the vast majority of cases, it's not worth the extra line. Consider the following example:

```
import re  
  
# These two lines ...  
regex = re.compile('Py...n')  
match = regex.search('Python is great')  
  
# ... are equivalent to ...  
match = re.search('Py...n', 'Python is great')
```

Don't get us wrong. Compiling a pattern once and using it many times throughout your code (e.g., in

a loop) comes with a significant performance benefit. In some anecdotal cases, compiling the pattern first lead to 10x to 50x speedup compared to compiling it again and again.

But the reason it is not worth the extra line is that Python's `re` library ships with an internal cache. At the time of this writing, the cache has a limit of up to 512 compiled regex objects. So for the first 512 times, you can be sure when calling `re.search(pattern, string)` that the cache contains the compiled pattern already.

Here's the relevant code snippet from `re`'s GitHub repository:

```
_cache = {} # ordered!

_MAXCACHE = 512
def _compile(pattern, flags):
    # internal: compile pattern
    if isinstance(flags, RegexFlag):
        flags = flags.value
    try:
        return _cache[type(pattern), pattern, flags]
    except KeyError:
        pass
    if isinstance(pattern, Pattern):
        if flags:
            raise ValueError(
```

```

        "cannot process flags argument with a
        ↪ compiled pattern")
    return pattern
if not sre_compile.isstring(pattern):
    raise TypeError("first argument must be
    ↪ string or compiled pattern")
p = sre_compile.compile(pattern, flags)
if not (flags & DEBUG):
    if len(_cache) >= _MAXCACHE:
        # Drop the oldest item
        try:
            del _cache[next(iter(_cache))]
        except (StopIteration, RuntimeError,
        ↪ KeyError):
            pass
    _cache[type(pattern), pattern, flags] = p
return p

```

Can you find the spots where the cache is initialized and used?

While in most cases, you don't need to compile a pattern, in some cases, you should. These follow directly from the previous implementation:

- You've got more than MAXCACHE patterns in your code.
- You've got more than MAXCACHE different patterns between two same pattern instances. Only

in this case, you will see “cache misses” where the cache has already flushed the seemingly stale pattern instances to make room for newer ones.

- You reuse the pattern multiple times. Because if you don’t, it won’t make sense to use sparse memory to save them in your memory. Even then, it may only be useful if the patterns are relatively complicated. Otherwise, you won’t see a lot of performance benefits in practice.

Compiling the pattern first and storing the compiled pattern in a variable for later use is often nothing but “premature optimization”—one of the deadly sins of beginner and intermediate programmers.

It doesn’t seem like a lot, does it? The intuition was that the real work is in finding the pattern in the text—which happens after compilation. And, of course, matching the pattern is the hard part. But a sensible compilation helps a lot in preparing the pattern to be matched efficiently by the regex engine—work that would otherwise have been done by the regex engine.

Regex’s `compile()` method does a lot of things such as:

- Combine two subsequent characters in the regex if both together indicate a special symbol such as certain Greek symbols.
- Prepare the regex to ignore uppercase and lowercase.
- Check for certain (smaller) patterns in the regex.
- Analyze matching groups in the regex enclosed in parentheses.

Note that all this work would have to be done by the regex engine at “matching runtime” if you wouldn’t compile the pattern first. If you can do it only once, it’s certainly a low-hanging fruit for performance optimizations – especially for long regular expression patterns.

21.4 Summary

You’ve learned about the `re.compile(pattern)` method that prepares the regular expression pattern—and returns a regex object which you can use multiple times in your code.

Video

Watch the short chapter video on YouTube:



<https://youtu.be/FkooNS7ZzjA>

Puzzle

```
import re

texts = ["I'm not happy.",
         "I don't want to dance.",
         "I don't love ice cream."]

pattern = re.compile("not |don't ")
print(pattern.sub('', texts[0]))
print(pattern.sub('', texts[1]))
print(pattern.sub('', texts[2]))
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



[https://app.finxter.com/learn/
computer/science/579](https://app.finxter.com/learn/computer/science/579)

— 22 —

Bonus Puzzles

In this final chapter, we want to leave you with some more puzzles. While these puzzles may not cover all regex topic that we discussed in this book, these puzzles will still give you a strong farewell touch of regex practicality.

We wish you good luck in solving these puzzles!

Puzzle 1

```
import re

text = '''
"Travelled" or "traveled".
Well, its "travelled" in Britian
and "traveled" in the US
'''

pattern_1 = 'traveled'
pattern_2 = 'travell?ed'

result_1 = re.findall(pattern_1, text,
↳ flags=re.IGNORECASE)
result_2 = re.findall(pattern_2, text,
↳ flags=re.IGNORECASE)

print(len(result_1))
print(len(result_2))
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



<https://app.finxter.com/learn/computer/science/580>

Puzzle 2

```
import re

text = '''
I don't always spell accommodation
correctly. Sometimes, when I am in a hurry,
I end up writing it like this: accomodate,
or acomodatin, or also accomomdatino."
'''

pattern = r'\b[acomodation]{7,20}\b'
matches = re.findall(pattern, text)

print(matches)
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



<https://app.finxter.com/learn/computer/science/581>

Puzzle 3

```
import re

text = "isRoomIlluminationTurnedOn"

pattern = r'([A-Z][a-z]+)'
replacement = r'_\1'
result = re.sub(pattern, replacement, text)

print(result.lower())
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



<https://app.finxter.com/learn/computer/science/582>

Puzzle 4

```
import re

target_folder = 'C:\\Users\\Python Enthusiast\\Data
↳ Science\\Learning\\Best Libraries\\NumPy'

pattern = r'.*\\([~\\]*)$'
replacement = r'\\1'
result = re.sub(pattern, replacement, target_folder)

print(result)
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



<https://app.finxter.com/learn/computer/science/583>

Puzzle 5

```
import re

text = '''
Harry Potter, Arthur Weasley,
Molly Weasley, Lily Potter,
Fred Weasley, Ron Weasley
'''

pattern = r'((\w+)\s+Weasley)+'
matches = re.findall(pattern, text)
for match in matches:
    print(match[1])
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



<https://app.finxter.com/learn/computer/science/584>

Puzzle 6

```
import re

target_folder = '''
Student rankings in exams:
1. Bob
2. Joe
3. Charlie
'''

pattern = r'(\d).*(\w+)\n'
replacement = r'\2 has rank \1\n'
result = re.sub(pattern, replacement, target_folder)

print(result)
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



<https://app.finxter.com/learn/computer/science/585>

Puzzle 7

```
import re

text = "Hmmm, that is the cat. These are the dogs."

pattern = r'(\w+)(\s+)(?=(is|are))\3(\s*)(.*?)\.'
replacement = r'\3\4\1\2\5?'
result = re.sub(pattern, replacement, text)

print(result.lower())
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



<https://app.finxter.com/learn/computer/science/586>

Puzzle 8

```
import re

python_code = '''
a = 1
b=2
if a==2:
    if (a >= 2) and (b ==2):
        a = a*2
        b = a-3
'''

pattern = r'a\s*=[^=]+?\n'
matches = re.findall(pattern, python_code)

for match in matches:
    print(match, end = '')
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



<https://app.finxter.com/learn/computer/science/587>

Puzzle 9

```
import re

python_code = '''
a = 1
b=2
if a==2:
    if (a >= 2) and (b ==2):
        a = a*2
        b = a-3
'''

pattern = r'a\s*=(?!=).*?\n'
matches = re.findall(pattern, python_code)

for match in matches:
    print(match, end = '')
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



<https://app.finxter.com/learn/computer/science/588>

Puzzle 10

```
import re

def get_string_info(str):
    pattern = r'.*?(\d+).*'
    replacement = r'\1'
    new_str = re.sub(pattern, replacement, str)
    return int(new_str)

list_of_strings = ['photo_1.png', 'photo_11.png',
↪ 'photo_129.png', 'photo_8.png', 'photo_21.png']

list_of_strings.sort()
print(list_of_strings)

list_of_strings.sort(key=get_string_info)
print(list_of_strings)
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



<https://app.finxter.com/learn/computer/science/589>

Puzzle 11

```
import re

m = re.match('.', 'Regex')
result = 'match' if m else 'nothing'

print(result)
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



<https://app.finxter.com/learn/computer/science/544>

Puzzle 12

```
import re

m = re.match('..', 'Y')
result = 'match' if m else 'nothing'

print(result)
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



<https://app.finxter.com/learn/computer/science/545>

Puzzle 13

```
import re

text = 'Speedy Gonzalez'
matches = re.findall('.', text)
result = len(matches) == len(text)

print(result)
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



<https://app.finxter.com/learn/computer/science/546>

Puzzle 14

```
import re

text = 'Spiderman'
matches = re.findall('...', text)
result = len(matches[2])

print(result)
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



<https://app.finxter.com/learn/computer/science/547>

Puzzle 15

```
import re

matches = re.findall('\.', 'Coconut')
result = len(matches)

print(result)
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



<https://app.finxter.com/learn/computer/science/548>

Puzzle 16

```
import re

string = 'Coconut'

c1 = re.search('C', string)
c2 = 'C' in string[0]
c3 = string.startswith('C')
c4 = 'C' == string[:1]

result = c1 and c2 and c3 and c4

print(result)
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



<https://app.finxter.com/learn/computer/science/549>

Puzzle 17

```
import re

string = 'Coconut'

m1 = re.search('.', string)
mlst2 = re.findall('.', string)

print(m1.group(0) == mlst2[0])
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



<https://app.finxter.com/learn/computer/science/550>

Puzzle 18

```
import re

regex = "e."
string = "Spiderman"
result = re.split(regex, string)

print(result[1])
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



<https://app.finxter.com/learn/computer/science/551>

Puzzle 19

```
import re

fruits = [
    "apple",
    "banana",
    "orange",
    "kiwi"
]

lx = lambda x: re.match("?.a", x)
result = list(filter(lx, fruits))

print(len(result))
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



<https://app.finxter.com/learn/computer/science/552>

Puzzle 20

```
import re

fruits = [
    "apple",
    "banana",
    "orange",
    "kiwi"
]

lx = lambda x: re.match("?.?a", x) != None
result = list(map(lx, fruits))

print(sum(result))
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



<https://app.finxter.com/learn/computer/science/553>

Puzzle 21

```
import re

fruits = [
    "apple",
    "banana",
    "orange",
    "bananas"
]

result = [x for x in fruits if re.fullmatch("b.+a",
↪ x)]

print(result[0])
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



<https://app.finxter.com/learn/computer/science/554>

Puzzle 22

```
import re

words = ['computer', 'car', 'cat', 'cow', 'crow',
        ↪ 'window']
result = [w for w in words if re.match('.*w$', w)]

print(len(result))
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



<https://app.finxter.com/learn/computer/science/555>

Puzzle 23

```
import re

regexs = ['^.*y$', '...', '[a-z]z+']

def number_of_matches(regex, string):
    return len(re.findall(regex, string))

result = max(regexs, key=lambda r:
    ↪ number_of_matches(r, 'clown'))
print(result)
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



<https://app.finxter.com/learn/computer/science/556>

Puzzle 24

```
import re

decode = {'112': 'S', '121': 'B', '211': 'P',
          '122': 'C', '221': 'E', '212': 'F',
          '222': 'G', '111': 'H', '1111': 'I',
          '1112': 'Y', '12': 'X', '1211': 'L',
          '2111': 'T', '1122': 'N', '1221': 'O'}

code = "0211011120211101110122101122
↪ 0110111101120110122012210122101211"
regex = "0([1|2]+)"

for m in re.finditer(regex, code):
    print(decode.get(m.group(1), ' '), end='')
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



<https://app.finxter.com/learn/computer/science/557>

Puzzle 25

```
import re

regex = '^((https?:\\/\|/)?(www\\.)?)' +
        '[a-zA-Z0-9]{1,256}\\.[a-zA-Z]{1,6}$'

urls = [
    'https://de.wikipedia.org',
    'nytimes.com',
    'https://www.google.de/?client=safari',
    'www.mywebsite.info',
    'https://www.google.de/?client=safari',
    'https://www.wayfair.de/v/checkout/basket'
]

rejected = [url for url in urls if not
    ↪ re.match(regex, url)]
print(len(rejected))
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



[https://app.finxter.com/learn/
computer/science/558](https://app.finxter.com/learn/computer/science/558)

Puzzle 26

```
import re

regex =
↪ r'[A-Z0-9a-z._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,64}'

emails = [
    'ronald.mcdonalds@mcd.com',
    '12345%0000.com',
    'fast!cars4sale@gmail.com'
    'me.myhouse.mycity@co.uk',
    '__main__@py.com',
    'one+two@equals.four',
    'the_sky_is_blue@in-my-universe.com',
    'a@b.c',
]

accepted = [email for email in emails if
↪ re.match(regex, email)]
print(len(accepted))
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



<https://app.finxter.com/learn/computer/science/559>

Puzzle 27

```
import re

regex = r'.{2,4}'
strings = [
    '$',
    '*p',
    '!§!',
    'x45o',
    '#v1~/'
]

result = next(s for s in strings if
    ↪ re.fullmatch(regex, s))
print(result)
```

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



<https://app.finxter.com/learn/computer/science/553>

Puzzle 28

Can you solve the puzzle?

Guess the output and check your solution for this puzzle on the Finxter app:



[https://app.finxter.com/learn/
computer/science/553](https://app.finxter.com/learn/computer/science/553)

Final Remarks

Congratulations, you made it through this whole regex workbook!

By reading this book, you have acquired a rare and precious skill: being able to read and write regular expressions in your own coding projects. You have worked through 19 chapters of content, videos, and puzzles—and, therefore, activated your ability to think *think in code*. Google’s former tech-lead Patrick Shyu argues that the understanding of regular expressions is among the top five programming skills overall. And he’s a practitioner who thrived in the largest computer science company on the planet.

Consistent effort and persistence is the key to success. If you feel that solving code puzzles has ad-

vanced your skills, make it a daily habit to solve one Python puzzle per day (and watch the related video on the [Finxter.com](https://finxter.com) web app). This habit alone will push your coding skills through the roof—and will ultimately provide you and your family a comfortable living in a highly profitable profession. Build this habit into your life—e.g., use your morning coffee break routine—and you will soon become one of the best programmers in your environment.

Where to go from here?

We publish a fresh code puzzle every couple of days on our website <https://finxter.com>. All puzzles are available for free. Our goal with Finxter is to make learning to code easy, individualized, and accessible.

- We worked hard to make this book as valuable for you as possible. But no book can reach perfection without feedback from early adopters and highly active readers. For any feedback, question, or problem you have, please send an email to admin@finxter.com.
- We highly appreciate your honest book review on your preferred bookseller (e.g., Amazon or Goodreads). We're not spending tons of money

on advertising and rely on loyal Finxters who spread the word. Would you mind sharing your learning experience with others?

- To grow your Python skills on autopilot, register for the free Python email course at the following URL: <https://blog.finxter.com/subscribe/>.
- Check out our popular Instagram account <https://www.instagram.com/codinglukas/> where we publish code lessons for free every day.
- You have now stretched your Python skills beyond the intermediate level. Why don't you start selling your skills and earn thousands of dollars while mastering the Python skill set? Practice is the best way to learn. If you're thinking about becoming a freelance developer, join the free *"How to Build Your High-Income Skill Python"* webinar replay at <https://blog.finxter.com/webinar-freelancer/>.

We're grateful that you have spent your time reading this book. Above everything else, we value your time. You'll receive a positive return on invested time and money: by applying your regular expression skills to the real world. Keep investing in yourself, start working on practical projects, and stay active within

the Finxter community for continuous improvement in Python.

More Python Textbooks

The Finxter “Coffee Break Python” textbook series helps you master computer science with a focus on Python coding. Please find the other textbooks in the following:

Coffee Break Python: 50 Workouts to Kickstart Your Rapid Code Understanding in Python.

The first bestselling book of the "Coffee Break Python" series offers 50 educative code puzzles, 10 tips for efficient learning, 5 Python cheat sheets, and 1 accurate way to measure your coding skills.



Get the ebook:

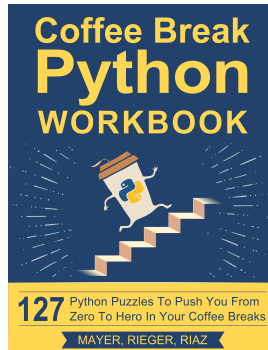
<https://blog.finxter.com/coffee-break-python/>

Get the print book:

<https://www.amazon.com/dp/B07GSTJPFD>

Coffee Break Python Workbook: 127 Python Puzzles to Push You From Zero to Hero in Your Coffee Breaks

This book is the chess grandmaster way of learning Python. 127 unique and brand-new Python puzzles - each pointing to gaps in your knowledge, challenges you to guess a solution and then explains potential solutions in an easy-to-understand manner.



Get the ebook:

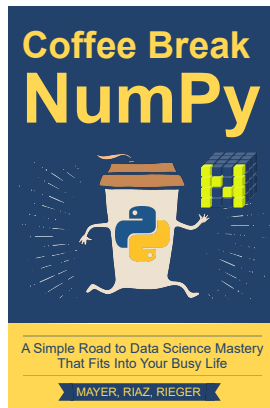
<https://blog.finxter.com/coffee-break-python-workbook/>

Get the print book:

<https://www.amazon.com/dp/B07WQR91PF>

Coffee Break NumPy: A Simple Road to Data Science Mastery That Fits Into Your Busy Life.

Coffee Break NumPy is a new step-by-step system to teach you how to learn Python's library for data science faster, smarter, and better. You simply solve practical Python NumPy puzzles as you enjoy your morning coffee.



Get the ebook:

<https://blog.finxter.com/coffee-break-numpy/>

Get the print book:

<https://www.amazon.com/dp/B07WHB8FWC>

Coffee Break Python Slicing: 24 Workouts to Master Slicing in Python, Once and for All.

Coffee Break Python Slicing is all about growing your Python expertise—one coffee at a time. The focus lies on the crucial slicing technique to access consecutive data ranges. Understanding slicing thoroughly is vital for your success as a Python developer.

As a bonus, you will track your individual Python coding skill level throughout the book.



Get the ebook:

<https://blog.finxter.com/coffee-break-python/>

Get the print book:

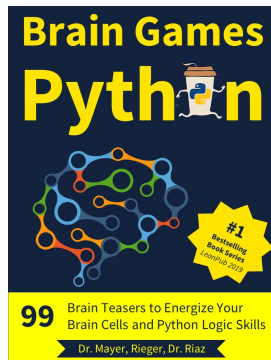
<https://www.amazon.com/dp/B07KSHLLG5/>

Brain Games Python: 99 Brain Teasers for Beginners to Energize Your Brain Cells and Python Logic Skills

Tired of Brain Fog? This Groundbreaking New Puzzle Book

- Revives Your Brain Cells
- Boosts Your Computational Intelligence
- Launches Your Programming Skills in Python

... Even If You Have Zero Programming Experience!



Get the ebook:

<https://blog.finxter.com/python-brain-games/>

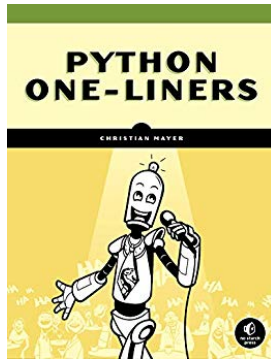
Get the print book:

<https://www.amazon.com/dp/B081NXFLZZ/>

Python One-Liners:

Python programmers will improve their computer science skills with these useful one-liners.

Python One-Liners will teach you how to read and write "one-liners": concise statements of useful functionality packed into a single line of code. You'll learn how to systematically unpack and understand any line of Python code, and write eloquent, powerfully compressed Python like an expert.



Get the print book:

<https://www.amazon.com/gp/product/B07ZY7XMX8>